CODEPOP ENGINEERING

TEHNIČKA BIBLIOTEKA / 01

SECRET CONTRACT OPERATOR

Projektovanje bezbednog Kubernetes operatora

Arhitektura · Go implementacija · GitOps · Produkcija



22 poglavlja / 30 razvojnih promptova
Praktične laboratorije i referentni Go primeri

IZDANJE 1.0

10. oktobar 2026. · Srpski / latinica

GO + KUBEBUILDER

Secret Contract Operator

Projektovanje bezbednog Kubernetes operatora

Izdanje 1.0 · 10. oktobar 2026.
Srpski jezik, latinica · Pripremljeno za Codepop

Jedinstvena projektna specifikacija i razvojni vodič za Go i Kubebuilder: validacija tajni, bezbednosne granice, status i zavisnosti, testovi, GitOps, instalacija i održavanje.

22	30	27
poglavlja	razvojnih promptova	primarnih izvora

Granica izdanja. Knjiga ne predstavlja već implementiran ili sertifikovan operator. Samostalan Go validator je odvojena laboratorija; pun operator, envtest, kind, ESO i deployment provere implementiraju se kroz razvojni plan.

PDF, EPUB i Markdown potiču iz istog izvornog teksta. U pratećem paketu su pojedinačna poglavlja, ilustracije, laboratorijski kod, promptovi i početni registar rada.

Svi credential primeri su sintetički. Nijedan primer ne koristiti kao produkcionu tajnu.

Sadržaj

Od prve specifikacije do proverljivog open-source release-a.

Predgovor: kako koristiti ovu knjigu 4

1. Problem koji rešavamo 6

2. Mesto u Kubernetes ekosistemu 8

3. Arhitektura i tok podataka 10

4. Bezbednosni model i autorizacija 13

5. Kanonski API i semantika ugovora 16

6. Repozitorijum i razvojno okruženje 20

7. Čisti Go validator 23

8. Reconcile petlja i pouzdan status 26

9. Watch, indeksi i performanse 29

10. Integracija sa External Secrets Operator-om 32

11. Provera potrošača i env konfiguracije 35

12. Opcioni injection i bezbedna izmena workload-a 37

13. Rotacija i kontrolisani rollout 39

14. CI, GitOps i stvarna kontrola deploy-a 42

15. Strategija testiranja i dokazivanje invarijanti 45

16. Metrike, događaji i operativni signal 48

17. Pakovanje, RBAC i instalacija 50

18. CI/CD, release i lanac isporuke 53

19. Operativne procedure i incidenti 55

20. API evolucija i open-source održavanje 57

21. Praktične laboratorije 59

22. Razvoj uz Codex i kontrola izvršavanja 62

Dodatak A. Trideset detaljnih Codex promptova 64

Dodatak B. Sažeta API i reason referenca 75

Dodatak C. Registar arhitektonskih odluka 78

Dodatak D. Rečnik i bibliografija 80

Predgovor: kako koristiti ovu knjigu

Secret Contract Operator je projekat Kubernetes operatora koji proverava da li aplikacija dobija tajne koje očekuje, u obliku koji ume da koristi. Knjiga prati put od početnog problema do dizajna API-ja, bezbedne implementacije, testiranja, pakovanja i održavanja javnog open-source projekta.

Ovo nije prepiska pretvorena u PDF. Raniji predlog operatora i razvojni promptovi ovde su objedinjeni u jednu tehničku specifikaciju. Nejasna ponašanja imaju eksplicitne odluke: šta tačno znači `Ready`, ko sme da kreira ugovor, šta se dešava pri neispravnoj rotaciji, kako se izbegava petlja status ažuriranja i zašto automatsko dodavanje promenljivih nije deo podrazumevanog režima.

Za koga je knjiga

Primarna publika su DevOps i platform inženjeri koji poznaju Deployment, Secret, namespace i osnovni RBAC. Za implementaciju je potrebno praktično znanje Go jezika, ali se složenost uvodi postepeno: prvo čisti validator bez Kubernetes zavisnosti, zatim kontroler, pa integracije i opcioni mehanizmi koji menjaju workload.

Čitalac koji želi da napravi prvi javni MVP treba da prati poglavlja 1–11, 15–20 i laboratoriju A. Poglavlja o injection-u, automatskom restartu i admission kontroli predstavljaju kasniju fazu. Održavalac klastera može prvo pročitati bezbednosni model, instalaciju, observability i operativne procedure.

Status materijala i granice provere

Knjiga je **projektna specifikacija i razvojni vodič, izdanje 1.0**, pripremljen 10. oktobra 2026. Ne predstavlja tvrdnju da je ceo operator već implementiran, objavljen ili bezbednosno sertifikovan. Predložena GitHub putanja, image repository i API grupa jesu projektna konfiguracija, a ne potvrda postojanja javnih release-ova.

Tri vrste primera su jasno razdvojene. **Izvršivi laboratorijski primer** je mali samostalan program iz pratećeg direktorijuma `examples/contractlab`. **Referentni isečak** prikazuje obrazac koji treba uklopiti u stvarni scaffold. **Specifikacija buduće funkcije** opisuje zahtev koji se implementira i prihvata tek posle testova. YAML uz novi CRD postaje primenljiv tek kada se implementira i instalira odgovarajuća šema.

Provera knjige obuhvata generisane formate, konzistentnost primera i lokalne testove izdvojenog Go validatora. Envtest, kind, stvarni ESO, Helm instalacija i rad u produkcionom klasteru ostaju razvojne provere; knjiga ih propisuje, ali ne predstavlja njihov nepostojeći rezultat.

Konvencije

Termin **tajna** označava osetljivu vrednost, a `Secret` Kubernetes objekat. **Ugovor** je instanca `SecretContract`. **Potrošač** je izabrani kontejner unutar workload-a. **Reconcile** je jedno usklađivanje opaženog i željenog stanja. Engleski identifikatori u kodu zadržani su radi kompatibilnosti sa Go i Kubernetes ekosistemom.

Sve vrednosti koje liče na lozinke u laboratorijama su sintetičke. Ne smeju se preneti u produkciju. Komande koje menjaju klaster izvršavaju se isključivo u namenskom test okruženju dok se ne završe provere iz release kontrolne liste.

Oznake `[S01](https://kubernetes.io/docs/concepts/configuration/secret/)`, `[S02](https://external-secrets.io/latest/api/externalsecret/)` i ostale upućuju na primarne izvore u bibliografiji. One potkrepljuju ponašanje Kubernetes-a i korišćenih alata. API ovog operatora, granice proizvoda, predloženi testovi i bezbednosne odluke predstavljaju **projektantske odluke ove knjige**, a ne zvaničnu Kubernetes specifikaciju.

Najvažnije ispravke početne ideje

`Ready=True` nije zabrana deploy-a niti dokaz da se aplikacija uspešno povezala na bazu. Promena `resourceVersion` nije dokaz da je lozinka rotirana. Annotation sa vremenom rotacije nije kriptografski dokaz starosti kredencijala. Zabrana logovanja vrednosti ne sprečava sama po sebi izvlačenje informacija kroz validaciona pravila.

Zbog toga je prva verzija operatora namenski mala: čita odobrene Secret objekte, proverava ugovor i objavljuje bezbedno, jasno ograničeno stanje. Sve što menja izvršavanje aplikacije dobija dodatnu autorizaciju, odvojen RBAC profil i posebne kriterijume prihvatanja.

1. Problem koji rešavamo

1.1 Od dostupne tajne do upotrebljive konfiguracije

Uobičajena platforma uspešno prenese tajnu iz AWS Secrets Manager-a, Vault-a ili drugog sistema u Kubernetes, ali aplikacija ipak ne uspe da se pokrene. Naziv ključa je promenjen, vrednost je prazna, JSON nije validan ili Deployment koristi stari Secret. Sinhronizacija je tehnički uspela; ugovor između konfiguracije i aplikacije nije.

Kubernetes podržava unos Secret podataka kroz `envFrom`, `secretKeyRef` i volumene. Obavezni, nedostupni Secret ili pojedinačni obavezni ključ mogu sprečiti pokretanje kontejnera. Međutim, `envFrom` ne zna da aplikacija očekuje dodatni ključ koji u Secret objektu uopšte ne postoji. Semantičku ispravnost sadržaja mora proveriti drugi sloj. SO1

Secret Contract Operator postavlja to pravilo eksplicitno. Umesto tribal knowledge-a i ručnog poređenja `.env.example` fajlova, tim dobija verzionisanu deklaraciju: aplikacija očekuje `DB_USERNAME`, neprazan `DB_PASSWORD` i validan JSON u `PROVIDER_CONFIG`.

1.2 Precizna vrednost proizvoda

Osnovni rezultat operatora nije nova kopija tajne nego **objašnjivo stanje ugovora**. Korisnik vidi koji zahtev nije ispunjen i koju generaciju ugovora je kontroler proverio. Pipeline može da proveri to stanje pre promene workload-a. Platform tim može da alarmira na regresiju posle rotacije, bez dodavanja SDK-a u svaku aplikaciju.

Dobar prvi korisnik je tim koji već koristi ESO i GitOps, a incidenti mu nastaju na granici između tajne i aplikacije. Loš prvi cilj je platforma koja želi centralni generator lozinki, izdavanje kredencijala za baze, revokaciju tokena i potpunu kontrolu nad aplikacionim restartima. To je drugi proizvod.

1.3 Šta operator garantuje

Za odobrenu referencu i uspešno pročitani snimak Secret objekta, operator proverava deklarisana pravila i beleži rezultat uz identitet tog opažanja. Ne piše stvarne vrednosti, njihove delove, njihove deterministične hash-eve ili parser greške koje mogu sadržati podatke u status, događaje ili metrike.

Ova garancija se odnosi na izlaze koje kontroliše implementacija. Ne može ukloniti tajnu iz memorije kube-apiserver-a, transporta pod TLS-om, Go heap-a ili prethodno konfigurisanog audit sistema. Bezbedan operator mora ograničiti te površine, a ne tvrditi da ne postoje.

1.4 Šta operator ne garantuje

Ugovor ne dokazuje da je lozinka prihvaćena na udaljenoj bazi. `minLength: 32` ne dokazuje entropiju. JSON validacija ne dokazuje da odgovarajući API key postoji kod provajdera. PEM dekodiranje ne dokazuje da je sertifikat pouzdan, vremenski važeći ili uparen sa privatnim ključem.

Ugovor ne kontroliše već pokrenut proces. Čak i posle uspešnog rollout-a aplikacija može imati sopstveni cache, pogrešno ime promenljive ili pogrešan connection pool. Aplikaciona provera spremnosti ostaje nezavisan signal.

1.5 Granice prvog izdanja

Predlog verzije `v0.1` obuhvata jedan lokalni Secret po ugovoru, pravila nad ključevima, stabilan status, opcionu read-only ESO proveru, proveru referenci u Deployment-u, StatefulSet-u i DaemonSet-u, metrike i namespace-scoped instalaciju. Nema kreiranja Secret objekata, kopiranja između namespace-ova, mutation webhook-a, mrežnih testova kredencijala ni automatskog brisanja podova.

Predlog `v0.2` dodaje strogo autorizovanu izmenu PodTemplate-a i restart signal. Admission zabrana deploy-a, agregacija više ugovora i podrška za kompleksne rollout kontrolere mogu doći kasnije. Oznake verzija ovde predstavljaju plan razvoja, ne postojeća izdanja.

Korisnička potreba	MVP odgovor	Nije obećano
Nedostaje ključ	`KeysValid=False` i naziv ključa	Popunjavanje tajne
Pogrešan JSON	Stabilna šifra `InvalidJSON`	Ispravljanje sadržaja
ESO nije spreman	Opciono blokiranje `Ready`	Popravka cloud pristupa
Deployment ne koristi Secret	`WorkloadsConfigured=False`	Promena workload-a u v0.1
Tajna je promenjena	Novo opažanje i validacija	Promena env-a aktivnog procesa

1.6 Kriterijum uspeha

Prvi release je koristan kada novi korisnik može da instalira operator sa čitljivim dozvolama, primeni tri mala manifesta i dobije tačnu dijagnozu bez čitanja implementacije. Nije uspešan samo zato što demonstracija radi za jedan ispravan Secret.

Za javni projekat traži se i negativan dokaz: nedozvoljena referenca nije pročitana, neispravna tajna nije izazvala restart, nepromenjen ugovor nije proizveo beskonačan tok status upisa i uklanjanje operatora nije obrisalo aplikacione podatke.

> **Kontrolna tačka.** Pre implementacije napiši jednu rečenicu koja razdvaja prenos tajne, validaciju ugovora i zdravlje aplikacije. Ako ih proizvod opisuje kao jednu stvar, obim još nije dovoljno precizan.

2. Mesto u Kubernetes ekosistemu

2.1 Kompozicija umesto dupliranja

External Secrets Operator prevodi deklaracije kao što je `ExternalSecret` u Kubernetes Secret objekte. Njegov `Ready` condition opisuje stanje sinhronizovanog ciljnog Secret-a, dok politike osvežavanja određuju kada se dobavljaju spoljne vrednosti. To je granica na kojoj naš operator preuzima proveru aplikacionog ugovora. S02

Secret Contract Operator ne dobija AWS ključeve, Vault tokene ili pristup spoljnim API-jima. Njegov mrežni minimum je Kubernetes API, uz opcioni zaštićeni metrics endpoint. Odvojene odgovornosti smanjuju broj uzroka incidenta koji jedna komponenta mora da razume.

2.2 Operator, admission kontrola ili CI skripta

Kubernetes operator obrazac kombinuje custom resource i kontroler koji usklađuje stanje. Pogodan je kada pravila treba ponovo proveriti i nakon početnog deploy-a, na primer posle promene zavisnog objekta. S03

CI skripta je jednostavnija ako treba samo jednokratno proveriti staging konfiguraciju. Međutim, ne reaguje sama na kasniju rotaciju. Admission webhook može odbiti upis objekta pre nego što se prihvati u API, ali ulazi u kritičnu putanju tog upisa. Operator sa statusom ostaje van te putanje, po cenu toga da rezultat nije automatska zabrana.

Projektna odluka je zato sledeća: operator održava stanje ugovora; CI/GitOps koristi stanje kao eksplicitnu kontrolnu tačku; sinhrono blokiranje ostaje zaseban, opcioni sloj. Time MVP ne zavisi od dostupnosti dodatnog webhook servera.

2.3 Go i Kubebuilder

Kubebuilder obezbeđuje scaffold Kubernetes API tipova, kontrolera, generisanja CRD-a i pratećih razvojnih artefakata. controller-runtime daje klijenta, manager, cache, event obradu i mehanizam reconcile-a. Njihove verzije moraju se uskladiti sa scaffold-om i Kubernetes bibliotekama. S04

Ne treba ručno kombinovati najnoviju pojedinačnu `k8s.io` biblioteku sa starijom controller-runtime minor verzijom samo zato što se import može rešiti. Verzija koja se bira za projekat postaje zaključana u `go.mod`, a prihvata se tek kada prođe lokalnu i CI matricu.

2.4 Predloženi identitet projekta

Element	Kanonska vrednost knjige
Naziv	Secret Contract Operator
Repozitorijum	`secret-contract-operator`

Element	Kanonska vrednost knjige
Predloženi Go module	`github.com/CodepopTech/secret-contract-operator`
Predložena API grupa	`secrets.codepop.tech`
Početna API verzija	`v1alpha1`
Kind / plural	`SecretContract` / `secretcontracts`
Kratko ime	`scn`
Annotation prefiks	`secrets.codepop.tech/`

Grupa `secrets.platform.io` iz ranije skice bila je primer. Za objavljivanje projekta treba koristiti DNS prostor koji održavalac kontroliše. Knjiga koristi predloženi Codepop namespace dosledno; ovo nije automatska migracija postojećeg klastera. Ako već postoji instaliran CRD sa drugom grupom, promena grupe stvara drugi API i zahteva plan kopiranja i validacije objekata.

2.5 Odluke koje izbegavaju rani tehnički dug

Jedan ugovor referencira jedan Secret u istom namespace-u. Referenca na ESO nije proizvoljan GVK, nego ograničen `ExternalSecret` iz podržanih API verzija. Workload reference sadrže eksplicitnu listu potrošača. Operator ne menja `ownerReferences` na tuđim Secret ili workload objektima.

Podrazumevana instalacija nema write dozvole nad workload-ima. Validation-only nije samo `if` u kodu, već i stvarni RBAC profil. Nema automatskog preuzimanja vlasništva nad aplikacijom i nema finalizer-a koji bi brisanje ugovora pretvorio u dostupnosni problem.

2.6 Alternativna rešenja

Ako aplikacija već ima pouzdanu startup validaciju, operator ne treba da je zameni nego da pruži raniju dijagnostiku. Ako se tajne konzumiraju isključivo kao fajlovi preko CSI-ja, ovaj MVP ne može da tvrdi da je proverio te vrednosti jer ih ne čita sa filesystem-a poda. Ako organizacija koristi obavezne policy engine kontrole, ugovor može biti dodatni signal, ali ne zaobilazi postojeće autorizacione granice.

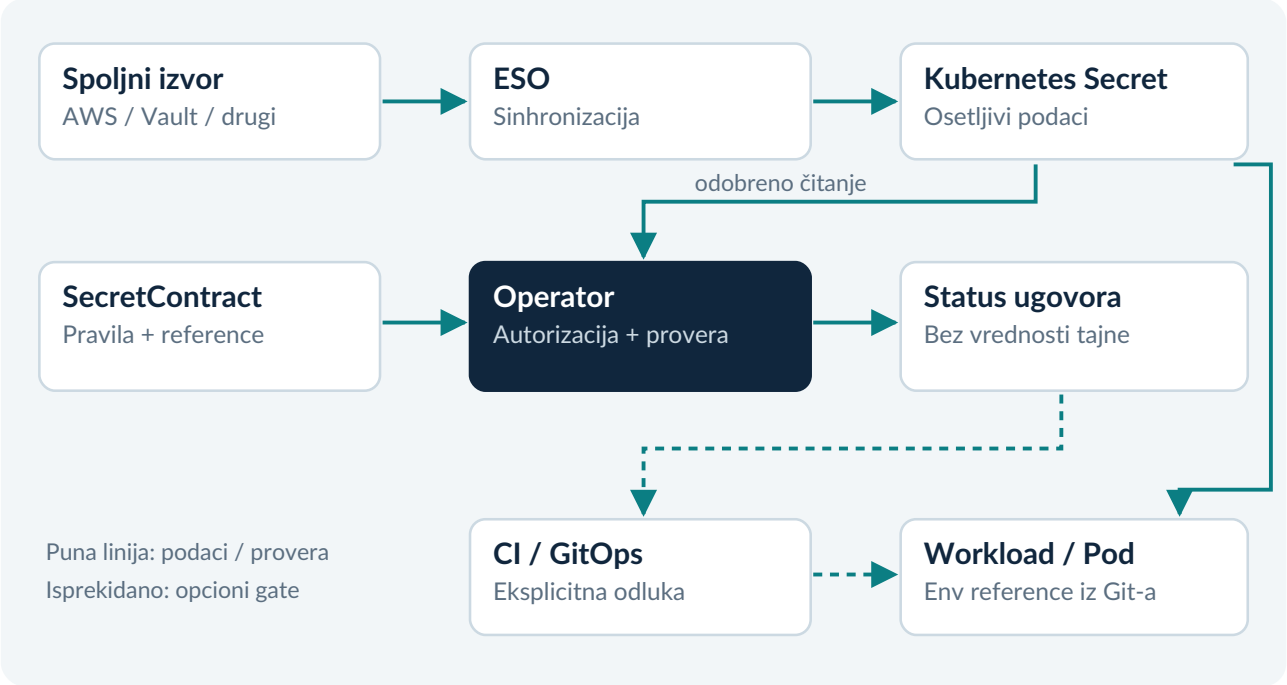
Nije dokazano da na tržištu ne postoji sličan projekat. Vrednost ovog dizajna ne zavisi od tvrdnje o potpunoj originalnosti, već od dobro ograničenog API-ja, poštenih garancija i pouzdanog rada pod opterećenjem.

> **Kontrolna tačka.** Pre prvog commit-a usvoji API grupu, minimalni podržani profil i listu funkcija koje izričito nisu MVP. To su arhitektonske odluke, ne detalji README-ja.

3. Arhitektura i tok podataka

3.1 Kontrolna ravan

Arhitektura je podeljena na sloj opažanja, deterministički evaluator i sloj objavljivanja rezultata. Sloj opažanja čita odobrene objekte. Evaluator ne poznaje Kubernetes mrežu i ne loguje. Sloj objavljivanja prevodi nalaz u ograničen status, metrike i događaje.



Slika 3.1 — Granice odgovornosti: sinhronizacija, validacija i pokretanje aplikacije.

U MVP-u nema toka Secret podataka ka Git-u, CI izlazu ili posebnoj bazi. Podaci postoje u Kubernetes Secret-u i kratkotrajno u memoriji operatora. Status sadrži metapodatke opažanja i šifre nalaza. Ova razlika mora biti vidljiva i u tipovima: rezultat validatora nema polje `Value`, `Actual`, `Input` ili `Snippet`.

3.2 Komponente

`api/v1alpha1` definiše javni API. `internal/contract` implementira proveru podataka. `internal/controller` upravlja reconcile tokom. `internal/status` uređuje stabilan rezultat. `internal/dependencies` čita ESO i workload stanje. `internal/authorization` proverava install-level opseg i odobrenja. `internal/telemetry` radi samo sa bezbednim rezultatima.

U kasnijoj fazi `internal/mutation` dobija zaseban interfejs. On prima plan promene koji sadrži reference i metapodatke, ali ne i Secret sadržaj. Takva granica smanjuje mogućnost slučajnog pretvaranja vrednosti u inline `env.value`.

3.3 Jedan reconcile kao transakcija opažanja

Kontroler učitava ugovor i proverava da li je u dozvoljenom namespace-u. Zatim proverava da je referenca dozvoljena pre čitanja sadržaja. Tek potom učitava Secret i beleži par `UID` i `resourceVersion`. Slede čista validacija, opcione provere zavisnosti i izračunavanje uslova.

`resourceVersion` u ovom operatoru tretiramo kao identifikator za proveru jednakosti, bez numeričke aritmetike. Aktuelna Kubernetes dokumentacija dopušta određena poređenja redosleda za isti API resource type uz propisane uslove; našem algoritmu to nije potrebno. Verzije različitih tipova resursa ne porede se kao zajednička transakcija. API podržava optimističku konkurentnost, ali ne pruža atomsku izmenu proizvoljnog Secret-a i Deployment-a. S05

Rezultat se poredi sa postojećim semantičkim statusom. Ako nema razlike, nema upisa. Ako razlika postoji, koristi se status patch sa odgovarajućom proverom konflikta. Kod konflikta se ponavlja čitanje i računanje, a ne uporno upisuje stari izračunati status.

3.4 Izvori događaja

Glavni događaji su promena generacije `SecretContract` objekta, promena ili brisanje referenciranog Secret-a, promena relevantnog workload-a i opcionog ESO objekta. Tajmer je potreban za starost rotacije i eventualnu polling integraciju. Bez tajmera ugovor može ostati zelen i nakon isteka maksimalne starosti, ako nijedan objekat nije promenjen.

Indeksi mapiraju ime zavisnosti na ugovore u istom namespace-u. Događaj iz jednog namespace-a ne sme probuditi ugovor koji koristi isto ime u drugom. Mapiranje mora biti zasnovano na namespace-u i imenu, dok opažanje razlikuje ponovno kreiranje objekta kroz njegov UID.

3.5 Cache kao svesna odluka

controller-runtime client može koristiti cache za čitanje, dok se upisi šalju direktno API serveru. Biblioteka pruža opcije za ponašanje cache-a i poseban ne-cache reader. Konfiguracija se proverava prema verziji projekta. S06

Jednostavan namespaced MVP može imati standardni Secret informer, ali tada cache čuva Secret sadržaj iz posmatranog opsega. To nije „samo metadata“. Ograničenje namespace-a smanjuje opseg, ali ne uklanja osetljivost procesa.

Stroži profil može posmatrati metapodatke i pojedinačno čitati odobrene Secret-e direktnim reader-om. Time se smanjuje trajno zadržavanje podataka u cache-u, ali se povećava broj API poziva i složenost watch implementacije. Samo dodavanje label selector-a cache-u nije bezbednosna granica: stvarne dozvole i dalje određuje RBAC.

3.6 Kontrola grešaka

Neispravan sadržaj je očekivani negativan rezultat, ne infrastrukturna greška. Za njega se postavlja `Ready=False`, bez agresivnog retry-ja. `NotFound` za Secret takođe je stanje zavisnosti. `Forbidden`, nedostupan API i greška discovery-ja predstavljaju različite infrastrukturne nalaze.

Spoljna greška se ne kopira automatski u javni status. Neki parseri i klijenti uključuju ulaz u poruku. Umesto toga koristi se stabilna šifra, na primer `DependencyAccessDenied`, uz lokalno formulisanu poruku bez originalnog payload-a.

> **Invarijanta arhitekture.** Podaci ulaze u validator, ali ne napuštaju njegovu granicu kao tekst dijagnostike. Ostatak sistema koristi samo kontrolisane identifikatore, brojače, vremenske oznake i reference.

4. Bezbednosni model i autorizacija

4.1 Zašto je ovo privilegovana komponenta

Operator koji čita Secret-e ima pristup stvarnim kredencijalima, čak i kada njegov README kaže „read-only“. Kubernetes bezbednosne smernice naglašavaju da `list` i `watch` nad Secret resursima mogu otkriti njihov sadržaj, kao i da kreiranje workload-a može biti put do tajni dostupnih u namespace-u. S07

Zbog toga rizik ne određuje samo broj RBAC glagola. Važno je koji namespace-ovi su obuhvaćeni, ko može da kreira ugovor, ko može da ga menja, ko vidi status i ko upravlja potrošačem kojem bi se tajna eventualno dodala.

4.2 Model pretnji

Akter ili kvar	Mogući problem	Projektna kontrola
Nepouzdan autor ugovora	Ispitivanje svojstava tuđe tajne	Ograniči autorstvo i odobrenja
Napadač sa izmenom workload-a	Iznošenje ubrizganih vrednosti	Odvojena mutation autorizacija
Greška u parseru	Vrednost u error poruci	Statične šifre, bez sirovih grešaka
Kompromitovan operator	Čitanje svih dozvoljenih tajni	Namespaced RBAC i manji opseg
Preveliki broj pravila	CPU i status opterećenje	Granice broja i veličine
Prekid API-ja	Zastareo pozitivan status	Provera generacije i opažanja
CI debug režim	Slučajan ispis podataka	Bez `set -x`, bez Secret dump-a

4.3 Validation oracle: curenje bez logovanja

Pretpostavimo da korisnik može da menja pravilo `pattern` nad tajnom koju inače ne sme da čita i da vidi rezultat. Takav korisnik može ponovljenim proverama zaključivati svojstva sadržaja. Problem nije nestao samo zato što status kaže „pravilo nije zadovoljeno“ i ne prikazuje vrednost.

I dužina, validnost određenog formata ili postojanje ključa mogu biti osetljivi. Ovo je zaključak iz dizajna sistema: svaki uslovni rezultat o tajni nosi informaciju. Zato u MVP-u pravo kreiranja i izmene ugovora imaju samo pouzdani administratori koji su već ovlašćeni za odgovarajući Secret. Čitaoci statusa dobijaju samo onaj obim metapodataka koji organizacija prihvata kao vidljiv.

Za više nepouzdanih tenant-a nije dovoljno dodati `allowed: true` u sam ugovor. Autor bi mogao sam sebe da odobri. Potrebna je zasebna, zaštićena grant politika kojom upravlja druga uloga i koja vezuje ugovor, Secret, dozvoljene vrste pravila i eventualni workload.

4.4 Zašto namespace nije cela autorizacija

Zabrana cross-namespace referenci sprečava jednostavnu klasu grešaka, ali svi korisnici unutar jednog namespace-a ne moraju imati jednaka prava. Dva tima u zajedničkom namespace-u mogu deliti aplikacionu infrastrukturu, a ne i tajne.

Predloženi sigurni početni profil koristi jedan namespace po granici poverenja i pouzdano autorstvo ugovora. Kada to nije prihvatljivo, prvo treba implementirati i testirati grant model. Tek nakon toga proizvod može tvrditi da podržava nepouzdane autore ugovora.

4.5 Dvostruko odobrenje za mutation

U kasnijem izdanju izmena workload-a zahteva da instalacija eksplicitno dozvoli mutation, da odgovarajući RBAC profil sadrži workload `patch`, da ugovor zahteva funkciju i da je cilj odobren od strane administratora workload-a. Jedna annotation koju svako može promeniti nije bezbednosna kontrola.

Zaštićeno odobrenje se vezuje za UID ugovora, ne samo za ime. Brisanje i ponovno kreiranje ugovora sa istim imenom ne nasleđuje automatski prethodno odobrenje. MVP mutation režima dopušta samo jedan ugovor koji upravlja promenama jednog workload-a; read-only ugovora može biti više.

4.6 Površine curenja

Secret vrednosti ne smeju završiti u strukturisanim log poljima, `%+v` ispisima objekata, panic payload-u, tracing atributima, HTTP dijagnostici, event porukama, metrikama, snapshot testovima ili CLI debug izlazu. Zabranjeni su i obični hash-evi sadržaja: za slabe ili predvidljive vrednosti takav hash može olakšati proveru pretpostavki.

Ne uključuj javni `pprof` endpoint. Heap dump sadrži stvarnu memoriju procesa i može obuhvatiti tajne. Go ne daje jednostavnu opštu garanciju pouzdanog brisanja svih kopija vrednosti iz heap-a. Smanji kopiranje, trajanje zadržavanja i broj objekata u cache-u, ali ne tvrdi „zeroization guaranteed“.

Kubernetes preporučuje enkripciju osetljivih podataka u mirovanju i ograničavanje pristupa. Base64 u manifestu nije enkripcija. Kod upravljanog klastera treba proveriti stvarnu konfiguraciju provajdera, a ne pretpostaviti isti default za svaku platformu. S08

4.7 Admission validacija sopstvenog API-ja

CRD šema treba da odbije duplirane ključeve, negativne dužine, prevelike liste i nedoslednu politiku. Međutim, API šema ne može dokazati da je autor ovlašćen da sazna svojstva proizvoljne druge tajne. To je zasebna autorizaciona odluka.

Operater koji koristi grant model proverava grant ponovo pri reconcile-u. Opozivanje odobrenja mora zaustaviti novo čitanje i novu mutation operaciju. Stari status treba zameniti sa `Ready=False`, razlogom `AccessNotApproved`, bez prethodnih detalja validacije koje novim pravima više ne treba prikazivati.

4.8 Bezbednosni prag za prvo javno izdanje

Pre objave moraju postojati testovi koji dokazuju da se neodobrena referenca odbija pre čitanja podataka, da svi bezbedni izlazi izostavljaju sentinel vrednost i njene uobičajene enkodirane oblike, da je mutation podrazumevano isključen i da se podaci ne pojavljuju pri greškama parsera.

Status i metrike imaju ograničen broj nalaza. Nazivi ključeva se tretiraju kao metapodaci koji ipak mogu biti interni. U strict režimu može se prikazivati samo broj neuspeha i razlog ugovora, ali ni takav režim ne daje nepouzdanim autorima pravo na neograničeno ispitivanje tajne.

> **Kontrolna tačka.** Bezbednosni review treba da odgovori: „Ko može da nauči nešto novo o tajni kroz ovaj operator?“ Odgovor „niko, jer ne logujemo“ nije dovoljan.

5. Kanonski API i semantika ugovora

5.1 API kao javno obećanje

Najskuplja greška u ranom operatoru često nije pogrešan algoritam nego nejasna semantika javnog polja. Dva boolean polja za istu odluku ili „pametni“ default koji menja značenje u različitim kontekstima otežavaju testiranje, migraciju i podršku.

Ova knjiga zato konsoliduje početnu skicu. Nedostajući Secret uvek onemogućava zadovoljen ugovor. Pravilo `nonEmpty` postoji na nivou ključa; ne postoji drugo globalno pravilo koje mu protivreči. Injection ima jedan režim umesto kombinacije `enabled` i `managed`. Tipovi sa default vrednošću `true` koriste pointer tamo gde je potrebno razlikovati odsutno polje od eksplicitnog `false`.

CRD koristi OpenAPI šemu, status subresource i list-map semantiku za uslove. Kubebuilder markeri generišu odgovarajuću šemu i olakšavaju održavanje. [S09]

5.2 Potpuni primer projektovanog API-ja

Sledeći manifest opisuje **ciljni API**, uključujući polja planirana za kasniju verziju. Za v0.1 injection ostaje `Disabled`, a restart `false`. Nije manifest za neki već instalirani operator treće strane.

```yaml
apiVersion: secrets.codepop.tech/v1alpha1
kind: SecretContract
metadata:
  name: payments-api
  namespace: payments
spec:
  secretRef:
    name: payments-api-secrets
  requiredKeys:
    - name: DB_USERNAME
      required: true
      nonEmpty: true
    - name: DB_PASSWORD
      required: true
      minLength: 16
      maxLength: 256
    - name: provider.json
      envName: PROVIDER_CONFIG
      required: true
      format: JSON
  externalSecretRef:
    name: payments-api-secrets
    apiVersion: external-secrets.io/v1
  workloadRefs:
    - apiVersion: apps/v1
      kind: Deployment
      name: payments-api
      containers: [api]
```

```
      includeInitContainers: false
      consumption: EnvVars
  injection:
    mode: Disabled
  rotation:
    restartOnChange: false
    maxAge: 720h
    rotatedAtAnnotation: secrets.codepop.tech/rotated-at
  policy:
    requireExternalSecretReady: true
    requireWorkloadReference: true
    requireRotationPolicy: false
```

5.3 Pravila nad ključevima

`requiredKeys` je lista jedinstvenih pravila po `name`, sa projektovanim maksimumom od 256. `required` i `nonEmpty` podrazumevano su `true`. Opciono odsutan ključ se preskače. Ako opciono polje postoji, sva njegova deklarisana pravila i dalje važe.

`minLength` i `maxLength` mere **bajtove dekodirane vrednosti**, ne broj Unicode znakova i ne dužinu base64 reprezentacije. `nonEmpty` znači nula bajtova; ne uklanja razmake niti automatski normalizuje sadržaj. Promena originalne vrednosti nije posao validatora.

`pattern` koristi Go regexp semantiku. Pravilo po podrazumevanom ponašanju traži poklapanje; za celu vrednost autor koristi sidra. `format` je `None`, `JSON`, `URI` ili `PEM`. JSON znači sintaktički ispravan JSON, uključujući skalarne vrednosti. URI znači apsolutni URI sa šemom; mrežni pristup nije dozvoljen. PEM znači jedan ili više ispravno dekodiranih blokova, bez drugog sadržaja između njih.

`envName` služi samo za eksplicitno env mapiranje. Secret ključ `provider.json` je validan ključ tajne, ali platforma može zahtevati prenosivo ime promenljive `PROVIDER_CONFIG`. Konzervativno pravilo za env ime je projektna odluka, ne tvrdnja da sve savremene Kubernetes verzije imaju isti skup dozvoljenih znakova.

5.4 Reference i politike

Sve reference su lokalne za namespace ugovora. `externalSecretRef` ima fiksan kind `ExternalSecret`; API verzija je iz install-level liste podržanih ESO verzija. Nema proizvoljnog čitanja bilo kog resursa preko korisničkog GVK-a.

`workloadRefs` podržava `apps/v1` Deployment, StatefulSet i DaemonSet. `containers` eksplicitno bira potrošače. Kada lista nije data u validation-only režimu, proveravaju se svi obični kontejneri. Init kontejneri uključuju se samo eksplicitno. Za mutation režim zahtevaj eksplicitnu listu kontejnera; time se izbegava nenamerno izlaganje tajne sidecar-u.

Politike `requireExternalSecretReady`, `requireWorkloadReference` i `requireRotationPolicy` odlučuju da li odgovarajući nalaz ulazi u agregatni `Ready`. Ako je politika obavezna, odgovarajuća konfiguracija mora postojati. Ne treba prihvatiti `requireExternalSecretReady: true` bez reference.

5.5 Uslovi i logika Ready

Uslovi su `SecretExists`, `KeysValid`, `ExternalSecretReady`, `WorkloadsConfigured`, `RotationPolicySatisfied` i agregatni `Ready`. Nepodešena opciona provera dobija `True` sa razlogom `NotConfigured`. Konfigurisana, ali još neproverena provera dobija `Unknown`.

Projektovana formula je:

```
Ready = authorized
    AND validSpec
    AND SecretExists
    AND KeysValid
    AND (not requireExternalSecretReady OR ExternalSecretReady)
    AND (not requireWorkloadReference OR WorkloadsConfigured)
    AND (not requireRotationPolicy OR RotationPolicySatisfied)
```

Pozitivna formula važi samo za uslove iz trenutne generacije ugovora. Infrastrukturna nemogućnost provere obaveznog uslova nije pozitivan rezultat. `Ready` može biti `Unknown` dok traju provere, odnosno `False` kada postoji određen negativan nalaz.

`WorkloadsConfigured=True` znači da je deklarisani PodTemplate pravilno referencirao Secret. Ne znači da su podovi pokrenuti ili da je aplikacija zdrava. Automatski restart, kada se doda, ima poseban status akcije; ne treba ga sakriti u značenje `Ready`.

5.6 Status jednog opažanja

```yaml
status:
  observedGeneration: 3
  observedSecret:
    name: payments-api-secrets
    uid: 53f81e06-1111-4444-8888-84fd83655102
    resourceVersion: "84721"
  lastCheckedTime: "2026-10-10T08:30:00Z"
  missingKeys: []
  violations: []
  missingKeyCount: 0
  violationCount: 0
  conditions:
    - type: KeysValid
      status: "True"
      observedGeneration: 3
      lastTransitionTime: "2026-10-10T08:30:00Z"
      reason: RulesSatisfied
      message: All declared key rules are satisfied.
```

Primer prikazuje deo statusa, ne kompletan izlaz. Svi condition zapisi imaju `observedGeneration`. Uslovi se tretiraju kao mapa po `type`, a ne kao lista u kojoj je redosled semantički važan. `missingKeys` i `violations` se sortiraju deterministički.

`lastCheckedTime` se ažurira pri novom relevantnom opažanju ili zakazanoj proveri, ne pri svakom prolasku kroz event queue. Promena vremena bez druge promene ne sme sama održavati reconcile petlju.

5.7 Ograničenja i tipovi

Za duration polja koristi se format koji podržava izabrani Go/Kubernetes tip: na primer `24h` ili `720h`, ne podrazumevani `30d`. Negativna i nulta maksimalna starost se odbijaju. Budući timestamp izvan dozvoljenog clock-skew prozora je greška metapodatka.

Ukupan broj status nalaza je ograničen, uz indikator skraćivanja. Pravila imaju ograničenje dužine pattern-a, a evaluator ograničenje veličine vrednosti i ukupnog obrađenog sadržaja. Te granice su deo verzionisane specifikacije i testova.

> **Kontrolna tačka.** Za svako polje napiši default, validaciju i posledicu na `Ready`. Ako to ne može da stane u jednu jasnu tabelu, API treba dodatno pojednostaviti.

6. Repozitorijum i razvojno okruženje

6.1 Reproduktivnost pre implementacije

Repozitorijum mora beležiti tačne verzije Go toolchain-a, Kubebuilder-a, controller-runtime-a, controller-gen-a, envtest binarnih fajlova i kind node image-a. Ova knjiga ne proglašava neproverenu kombinaciju „najnovijom kompatibilnom“. Kompatibilnost nastaje iz izabranog scaffold-a i rezultata testova.

Početni `toolchain-report.txt` generiše se lokalno, ali bez kubeconfig sadržaja, environment dump-a ili kredencijala. Dovoljni su `go version`, `kubebuilder version`, `kubectl version --client`, `helm version` i hash odgovarajućih konfiguracionih fajlova.

6.2 Bootstrap komande

Sledeće komande su predlog za prazan repozitorijum. Ne pokreću se preko postojećeg projekta bez prethodnog pregleda.

```
mkdir secret-contract-operator
cd secret-contract-operator

git init
kubebuilder init \
  --domain codepop.tech \
  --repo github.com/CodepopTech/secret-contract-operator

kubebuilder create api \
  --group secrets \
  --version v1alpha1 \
  --kind SecretContract \
  --resource --controller

make generate
make manifests
go test ./...
```

Ako lokalna Kubebuilder verzija zahteva drugačiji raspored zastavica ili dodatni izbor plugina, prati njen `--help` i zabeleži izbor. Ne menjaj nasumično Go module da bi komanda prošla. Sačuvaj `PROJECT` fajl i generisani Makefile kao deo scaffold ugovora. S04

6.3 Struktura projekta

```
secret-contract-operator/
  AGENTS.md
  api/v1alpha1/
  cmd/
  internal/
    authorization/
    contract/
```

```
    controller/
    dependencies/
    mutation/
    status/
    telemetry/
  config/
    crd/
    manager/
    rbac/
    samples/
  charts/secret-contract-operator/
  examples/
  test/e2e/
  docs/adr/
  prompts/
  tracking/
```

Ne otvaraj svaki prazan paket odmah. `mutation` može nastati tek u drugoj fazi. Važnije je da čisti validator nema slučajnu zavisnost od event recorder-a ili cloud klijenta nego da struktura izgleda veliko.

6.4 AGENTS.md kao inženjerski ugovor

Codex podržava projektne instrukcije u `AGENTS.md`; hijerarhija instrukcija omogućava opšti kontekst i specifičnije smernice po direktorijumu. Takav fajl je korisniji od ponavljanja celog konteksta u svakom zadatku. S25

Za ovaj projekat root instrukcija propisuje granice funkcija, zabranu curenja, isti namespace za reference, bez mutation dozvola u MVP-u i obavezne testove. Ne sadrži prave tajne, produkcione endpoint-e ili pravilo koje dozvoljava automatski deploy u stvarni klaster.

```
Read docs/specification.md before changing public API.
Read docs/security.md before touching Secret handling.
Never serialize Secret objects to diagnostics.
Default profile is validation-only, including RBAC.
Do not overwrite unrelated work or existing env entries.
Record executed tests; never mark an unrun test as passed.
```

6.5 Razvojne provere

`gofmt` i `go vet` proveravaju osnovnu higijenu, ali nisu dokaz ispravnog reconcile-a. `make generate` obnavlja DeepCopy kod. `make manifests` obnavlja CRD i RBAC artefakte. CI mora potvrditi da posle generisanja nema diff-a.

`go test ./...` ne sme neprimetno zahtevati stvarni produkcioni klaster. Unit testovi i envtest se razdvajaju po jasnom test setup-u. E2E protiv kind-a pokreće se posebnom metom i namenskim kubeconfig-om.

6.6 Fake cluster nije isto što i envtest

Fake controller-runtime client koristan je za određene unit testove, ali ne predstavlja API server sa svim pravilima podrazumevanja i status subresource-a. Envtest pokreće API server i etcd, bez kubelet-a i standardnih workload kontrolera. Zbog toga u envtest-u ne treba čekati da Deployment zaista napravi podove. S10

Prvi cilj je mali broj jasnih testova: create ugovora, provera statusa, promena Secret-a i no-op drugi reconcile. Tek posle toga se dodaju opcione integracije i full-cluster testovi.

> **Kontrolna tačka.** Čist checkout treba da ima dokumentovan put do unit testova bez cloud kredencijala. Sve što zahteva mrežu, preuzimanje alata ili lokalni klaster mora biti eksplicitno navedeno.

7. Čisti Go validator

7.1 Granica najosetljivijeg koda

Validator je jedina komponenta koja treba da razume sadržaj deklarisanih Secret ključeva. Prima pravila i `map[string][]byte`, a vraća strukturisani rezultat bez osetljivog ulaza. Nema logger, event recorder, Kubernetes client, HTTP client ili sat.

Takva organizacija omogućava brze table-driven testove i fuzz testiranje. Takođe olakšava review: svaki izlaz iz funkcije može se pregledati kao potencijalni kanal curenja.

7.2 Dva nivoa greške

Neispravna specifikacija znači da pravilo nema smisla: isti ključ je naveden dva puta, regexp ne može da se kompajlira ili je minimum veći od maksimuma. **Neispravni podaci** znače da dobro definisano pravilo nije zadovoljeno.

Ove slučajeve nemoj mešati. `InvalidPattern` je greška konfiguracije ugovora, a `PatternMismatch` nalaz o podacima. Oba moraju biti bezbedna za prikaz, ali odgovornost za ispravku nije ista.

7.3 Tipovi bez polja za vrednosti

```
// Referentni isečak: bez Kubernetes zavisnosti.
type Rule struct {
    Name      string
    Required  *bool
    NonEmpty  *bool
    MinLength *int
    MaxLength *int
    Pattern   string
    Format    string
}

type Violation struct {
    Key    string `json:"key,omitempty"`
    Rule   string `json:"rule"`
    Reason string `json:"reason"`
}

type Result struct {
    SpecValid  bool        `json:"specValid"`
    Valid      bool        `json:"valid"`
    Missing    []string    `json:"missing"`
    Violations []Violation `json:"violations"`
}
```

Korisnički tekst poruke nije deo pravila. Time se sprečava da neko unese tajnu u `messageTemplate`, a da je operator kasnije nekritički prenese u status. Javni tekst poruke generiše se iz poznate šifre u posebnom sloju.

7.4 Redosled evaluacije

Prvo se proveravaju granice i ispravnost pravila. Zatim se za svaki ključ utvrđuje da li postoji. Ako je odsutan i opcion, provera tog ključa se završava. Ako je odsutan i obavezan, dodaje se `MissingRequiredKey`, bez pokretanja parsera nad praznim ulazom.

Za prisutnu vrednost proverava se budžet veličine, pa praznina, dužina, pattern i format. Rezultat se sortira po ključu i vrsti pravila. Više nalaza za isti ključ je dozvoljeno, ali treba ograničiti ukupan broj nalaza i izbeći beskorisno ponavljanje.

Ne skraćuj vrednost pre validacije da bi izbegao budžet. Skraćivanje bi promenilo značenje provere. Umesto toga vrati `ValueTooLarge` ili `EvaluationBudgetExceeded` i jasno dokumentuj taj razlog.

7.5 Regularni izrazi

Go regexp implementacija ima garantovano vreme izvršavanja linearno u veličini ulaza za svoje podržane izraze. To je dobar razlog da se ne uvodi drugi engine samo radi dodatnih konstrukcija. I dalje treba ograničiti broj pravila, veličinu ulaza i dužinu pattern-a jer ukupni posao raste sa njihovim proizvodom. S11

Ne vrati direktno `regexp.Compile` grešku u status. Pattern je deo ugovora, ali može sadržati osetljiv ili neprikladan tekst. Dovoljno je `InvalidPattern`, uz putanju do pravila. U dokumentaciji objasni sidra i testiraj razliku između substring i full-string očekivanja.

7.6 JSON, URI i PEM

Za JSON sintaksu je dovoljan `json.Valid`. Za semantički ugovor po JSON poljima bila bi potrebna nova, eksplicitna funkcija sa kontrolisanom šemom i novim granicama kompleksnosti. MVP ne obećava JSON Schema validaciju.

URI parser mora potvrditi prisustvo šeme, ali ne sme tražiti host za sve validne URI oblike: `urn:` i drugi apsolutni URI-ji ne moraju imati mrežni authority. Ne otvaraj adresu i ne radi DNS lookup. Time validacija ostaje deterministička i ne postaje SSRF površina.

PEM provera dopušta jedan ili više blokova sa nepraznim dekodiranim sadržajem, uz razmake između blokova. Pre svakog `pem.Decode` poziva proveri očekivani početak, jer decoder može preskočiti određeni ne-PEM tekst. Ne prikazuj sadržaj bloka pri grešci. Ovo nije X.509 verifikacija.

7.7 Samostalna laboratorija

Prateći direktorijum `examples/contractlab` sadrži implementaciju ovog uskog evaluator-a i njegove testove. To nije ceo operator. Kod koristi standardnu biblioteku, pa se može proveriti nezavisno od Kubernetes scaffold-a.

```
cd examples/contractlab
go test ./...
go test -race ./...
go vet ./...
```

Testovi pokrivaju obavezne i opcione ključeve, praznu vrednost, byte dužinu, pogrešan regexp, JSON, apsolutni URI, PEM, duplirana pravila, budžet i sentinel curenje. Namerno postoje sintetičke osetljive vrednosti u test fixture-ima; zabrana se odnosi na njihove javne izlaze, ne na nemogućnost testiranja tajne.

7.8 Prenos u pravi operator

Adapter između CRD tipova i čistog evaluator-a eksplicitno normalizuje default vrednosti. Ne oslanja se na to da je API server sigurno primenio default, jer unit testovi mogu konstruisati objekte direktno u Go-u. Isto pravilo važi za stare ili nepotpune objekte.

Pre prenosa u produkcioni kontroler dodaj fuzz test za proizvoljne bajtove i kombinacije pravila. Fuzzing traži panic, nedeterminističnost i neočekivani izlaz; ne dokazuje odsustvo svake ranjivosti. Go ima ugrađen tok za coverage-guided fuzz testove. S12

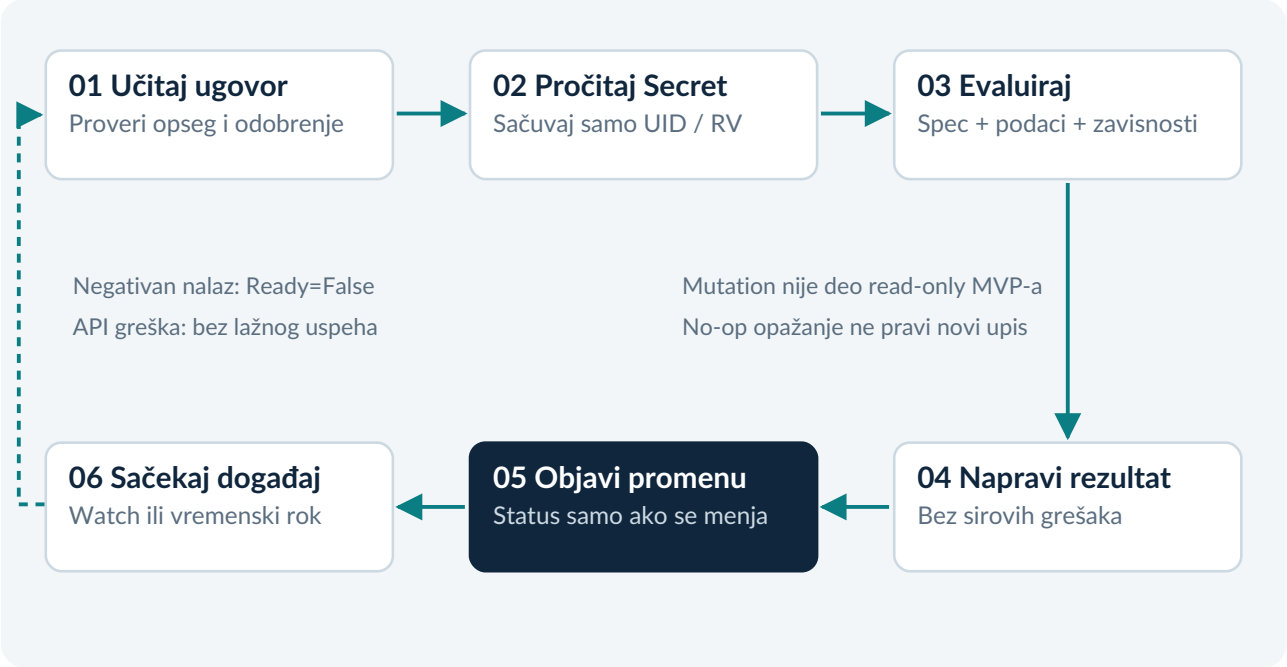
> **Kontrolna tačka.** Validator mora moći da se testira bez Kubernetes-a, i njegov rezultat mora moći bezbedno da se JSON-serializuje čak i kada je ulaz namerno neprijateljski.

8. Reconcile petlja i pouzdan status

8.1 Reconcile nije obrada pojedinačnog događaja

Kontroler ne sme pretpostaviti da će dobiti svaki međukorak između dve promene. Reconcile radi sa trenutnim opaženim stanjem i mora biti idempotentan. Dve identične evaluacije treba da proizvedu isti rezultat i nula dodatnih workload promena.

Praktičan tok je: učitaj ugovor; proveri autorizaciju i spec; pročitaj zavisnosti; izračunaj rezultat; izračunaj sledeće vreme provere; objavi samo stvarnu promenu. Samo opcioni mutation modul, kada je eksplicitno dozvoljen, umeće plan promene pre završnog statusa.



Slika 8.1 — Reconcile tok sa razdvajanjem negativnog nalaza, infrastrukturne greške i opcionih promena.

8.2 Razlikovanje NotFound i Forbidden

Ako ugovor više ne postoji, reconcile završava bez greške i čisti sopstvene metričke serije. Ako Secret ne postoji, postavlja `SecretExists=False`; ne označava to kao panic ili neograničen retry. Watch na kreiranje Secret-a kasnije ponovo pokreće proveru.

Ako API vrati `Forbidden`, ne sme se prikazati poruka „Secret ne postoji“. To prikriva problem instalacije ili autorizacije. Koristi odvojeni razlog `SecretAccessDenied`, bez objavljivanja sirovog odgovora servera.

Ako je čitanje neuspešno zbog timeout-a, stari pozitivan nalaz ne treba predstavljati kao novo uspešno opažanje. `observedSecret` ostaje identitet prethodnog stvarnog opažanja ili se čisti prema dokumentovanoj politici; trenutni obavezni uslov nije potvrđen.

8.3 Uslovi i observedGeneration

Svaki rezultat mora se vezati za generaciju specifikacije koju je kontroler proverio. Ako je ugovor promenjen između čitanja i upisa, novija generacija ne sme dobiti stari pozitivan rezultat kao da je obrađena.

Status subresource odvaja izveštavanje kontrolera od korisničkog specifikacionog dela. CRD-ovi ga podržavaju, a generated schema mora eksplicitno da ga uključi. S13

```
// Referentni isečak, uklopiti u postojeći status builder.
meta.SetStatusCondition(&next.Conditions, metav1.Condition{
    Type:               "KeysValid",
    Status:             metav1.ConditionTrue,
    ObservedGeneration: contract.Generation,
    Reason:             "RulesSatisfied",
    Message:            "All declared key rules are satisfied.",
    LastTransitionTime:  metav1.NewTime(now),
})
```

`lastTransitionTime` se menja pri promeni condition statusa, ne kao heartbeat. Helper i status builder moraju sačuvati prethodnu tranziciju kada je status isti. Promena `reason` ili opažene generacije može biti relevantna, ali nije isto što i prelaz iz `False` u `True`.

8.4 Semantički no-op

Najčešća petlja nastaje kada reconcile uvek postavi `lastCheckedTime=now`, upiše status i zatim dobije novi događaj zbog sopstvenog upisa. Predicate koji ignoriše status promene može pomoći, ali ne opravdava bespotrebne API upise.

Napravi status builder koji stabilno sortira nalaze, uklanja zastarele greške i čuva vremenske oznake. Tek posle poređenja semantičkog sadržaja dodaj novo vreme kada je stvarno obrađen novi ulaz ili zakazana vremenska granica.

Test treba da meri broj status upisa. Posle početnog usklađivanja, dodatni reconcile bez promene zavisnosti mora imati nula status patch poziva.

8.5 Optimističko zaključavanje

```
// Referentni isečak: ne izostavljati ponovni obračun pri konfliktu.
base := current.DeepCopy()
current.Status = desiredStatus
patch := client.MergeFromWithOptions(
    base,
    client.MergeFromWithOptimisticLock{},
)
if err := r.Status().Patch(ctx, current, patch); err != nil {
    return ctrl.Result{}, err
}
```

controller-runtime podržava merge patch sa opcijom optimističkog zaključavanja. To omogućava odbijanje zastarelog upisa umesto tihog prepisivanja konkurentne izmene. Potpis i ponašanje proveri prema zaključanoj verziji biblioteke. S06

Povratak greške kontroliše retry kroz work queue. Ne dodaj paralelnu beskonačnu petlju koja ponavlja isti stari patch. Pri ponovnom reconcile-u ponovo se čitaju podaci i računa status.

8.6 Requeue i vreme

Za neispravan sadržaj koristi događaje zavisnosti. Za proveru maksimalne starosti zakaži sledeći relevantni rok. Za opcioni polling ESO statusa koristi umeren interval sa jitter-om. Za prolaznu API grešku koristi postojeći mehanizam backoff-a, bez dodatnog agresivnog timer-a.

Najkraći od potrebnih pozitivnih rokova postaje `RequeueAfter`. Negativni izračunati interval znači da je granica već pređena: odmah ažuriraj nalaz, a sledeću proveru zakaži uz mali definisani minimum, ne beskonačnu petlju sa nultim čekanjem.

8.7 Brisanje i vlasništvo

MVP nema spoljne objekte koje mora da očisti i zato mu finalizer nije potreban. Brisanje ugovora ne briše Secret, Deployment, grant ili ESO objekat. Operator ne preuzima njihove owner reference-e.

Ako kasnija mutation verzija ostavi annotation ili env referencu, politika uklanjanja mora biti eksplicitna. Bezbedan početni izbor je da brisanje ugovora zaustavi dalje upravljanje, ali ne uklanja aplikacionu konfiguraciju automatski. Takvo uklanjanje može srušiti servis.

> **Kontrolna tačka.** Tri uzastopna reconcile-a bez promene treba da ostave identičan status i iste workload metapodatke. To je osnovni test kvaliteta kontrolera.

9. Watch, indeksi i performanse

9.1 Od zavisnosti do ugovora

Secret promena treba da pokrene samo ugovore koji taj Secret koriste. Listanje svih ugovora i ručno filtriranje za svaki događaj radi u maloj demonstraciji, ali stvara nepotreban posao u većem klasteru.

Indeks `spec.secretRef.name` registruje se pri pokretanju manager-a. Za događaj Secret-a kontroler lista ugovore sa odgovarajućim poljem u istom namespace-u. Za workload indeks koristi složen ključ kao `apps/v1|Deployment|payments-api`, uz namespace kao zasebno ograničenje.

```
// Referentni isečak. Naziv polja je privatna konstanta indeksa.
const secretNameIndex = "contract.secretName"

err := mgr.GetFieldIndexer().IndexField(
    ctx,
    &apiv1.SecretContract{},
    secretNameIndex,
    func(obj client.Object) []string {
        c := obj.(*apiv1.SecretContract)
        if c.Spec.SecretRef.Name == "" {
            return nil
        }
        return []string{c.Spec.SecretRef.Name}
    },
)
```

Pripadajući map handler ne loguje Secret objekat. Ako indeksirano listanje ne uspe, beleži bezbedan operativni razlog i ne pokušava neograničeno cluster-wide pretraživanje kao tihi fallback.

9.2 Owned i ne-owned resursi

Secret i Deployment nisu objekti koje ovaj operator poseduje. Nemoj dodati `Owns(Secret)` i očekivati ispravno mapiranje ako owner reference ne postoji. Za spoljne zavisnosti koristi eksplicitni watch i map funkciju.

Kubebuilder dokumentuje watch i predicate mehanizme za filtriranje događaja i usmeravanje reconcile-a. Ključno je izabrati model koji odgovara stvarnom vlasništvu nad objektima. S14

9.3 Predicate nije univerzalni filter

Za primarni CR korisno je ignorisati promene samo statusa i reagovati na generaciju. Ali isti `GenerationChangedPredicate` ne sme automatski biti primenjen na Secret događaje: Secret promene ne moraju imati istu generation semantiku kao specifikacija CR-a.

Za Secret prati relevantnu promenu verzije, kreiranje i brisanje. Za workload posmatraj PodTemplate i relevantna odobrenja, ali sopstvenu bezazlenu status promenu workload kontrolera nemoj pretvoriti u intenzivnu petlju.

Ako autorizacija zavisi od annotation-a ili grant resursa, njihov update mora biti watch okidač. Filter koji reaguje samo na `.spec` mogao bi propustiti opoziv prava.

9.4 Metadata-only profil

controller-runtime cache ima konfiguracione mogućnosti za tipove i namespace opsege. Njihova upotreba utiče i na memoriju i na konzistentnost čitanja. S15

Za hardened profil metadata watch nije isto što i čitanje punog Secret-a pa brisanje `Data` posle logovanja. Osetljivi podaci nikada ne treba da uđu u opšti cache koji nije potreban evaluator-u. Direktni reader koristi se nakon provere odobrenja i ne ostavlja podatke u dodatnom application-level cache-u.

Ovaj profil zahteva konkretne testove za izabranu biblioteku: koji objekat event handler dobija, da li slučajni `Get` preko cached client-a može pokrenuti full informer i kako se ponaša ponovno povezivanje watch-a. Bez takvih testova ne tvrdi da je cache „bez tajni“.

9.5 Budžet opterećenja

Ako se `N` ugovora proverava polling-om svakih `T` sekundi, gruba osnovna učestalost je `N/T` evaluacija u sekundi, pre retry-ja i stvarnih promena. Za 10.000 ugovora i interval od 300 sekundi to je približno 33,3 evaluacije u sekundi. To je račun za planiranje, ne izmereni rezultat implementacije.

Ako svaki ugovor pri tome direktno čita tri zavisnosti, broj API zahteva raste proporcionalno. Indeksi ne uklanjaju taj posao; oni uklanjaju nepotrebno traženje potrošača kada stigne događaj. Vremenske provere treba zakazivati prema roku isteka, uz ograničen rezervni polling gde je potreban.

9.6 Backpressure i fer raspodela

Postavi ograničen broj paralelnih reconcile radnika. Testiraj da li jedan Secret sa mnogo ugovora može potisnuti ostale. Jitter sprečava da svi ugovori nakon restarta istovremeno ponove vremensku proveru.

Ne drži globalni mutex dok čitaš API. Za kratke strukture cache-a i ograničenja koristi minimalne kritične sekcije. Za jedan ugovor kontroler treba da bude bezbedan i kada se nakon greške isti ključ brzo vrati u red.

9.7 Merenje bez lažnih obećanja

Benchmark izveštaj beleži broj namespace-ova, broj ugovora, broj ključeva, veličine vrednosti, promene u sekundi, cache profil, CPU i memorijske limite i tačne verzije alata. Meri P50/P95/P99 latenciju do novog statusa, API zahteve i broj status upisa.

Ciljevi poput „P95 ispod pet sekundi“ mogu biti početni acceptance prag, ali se ne objavljuju kao performanse dok nisu izmereni. Ne zamenjuj simulirani workload produkcionim benchmark-om.

> **Kontrolna tačka.** Promena jednog Secret-a treba da izazove očekivan broj zavisnih reconcile-a, a ne celo-klastersko ponovno računanje. Izmeri i drugi reconcile bez promene.

10. Integracija sa External Secrets Operator-om

10.1 Integracija koja ostaje opciona

Klaster bez ESO-a mora moći da pokrene validation-only operator i koristi obične Kubernetes Secret-e. Zato startup ne sme bezuslovno da čeka informer za CRD koji možda nije instaliran.

Prva implementacija može koristiti discovery podržanih ESO verzija i ograničen polling samo kada ugovor ima `externalSecretRef`. Naprednija verzija registruje watch kada je CRD prisutan, uz ponovno otkrivanje ili dokumentovan restart kada se ESO naknadno instalira. Obe strategije su prihvatljive ako su eksplicitne i testirane.

10.2 Unstructured čitanje sa fiksnim opsegom

```
// Referentni isečak: apiVersion prethodno prolazi allowlist.
es := &unstructured.Unstructured{}
es.SetAPIVersion(ref.APIVersion)
es.SetKind("ExternalSecret")
err := r.APIReader.Get(ctx, client.ObjectKey{
    Namespace: contract.Namespace,
    Name:      ref.Name,
}, es)
```

Unstructured pristup izbegava vezivanje za ESO Go API paket, ali ne uklanja potrebu za šemom i kompatibilnošću. Kod mora bez panic-a obraditi odsutne `status.conditions`, pogrešan tip polja i nedostupan API. Ne sme koristiti korisnički zadat neograničen kind ili API group.

10.3 Šta proveravamo

Proverava se da referencirani ESO objekat postoji, da cilja isti Secret koji ugovor validira i da odgovarajući `Ready` condition ima vrednost `True`. Efektivni target se računa prema podržanoj ESO šemi, uključujući njen dokumentovani default kada `spec.target.name` nije postavljen.

Ako healthy ExternalSecret proizvodi potpuno drugi Secret, ugovor mora prijaviti `ExternalSecretTargetMismatch`. Samo poklapanje imena `ExternalSecret` objekta nije dovoljan dokaz cilja.

Ne kopiraj ESO `message` ili proizvoljni `reason` doslovno. Koristi lokalne razloge `ExternalSecretNotReady`, `ExternalSecretNotFound`, `ExternalSecretAPINotAvailable` i `ExternalSecretTargetMismatch`. Nepoznati schema oblik je `UnsupportedExternalSecretSchema`, ne automatski `Ready=True`.

10.4 Svežina i starost nisu isto

ESO `refreshTime` odnosi se na dobavljanje i osvežavanje ciljnog Secret-a, a `Ready` stanje ima svoje definisano značenje sinhronizacije. To nije dokaz vremena promene osnovnog kredencijala kod provajdera. SO2

Ako je isti DB password uspešno sinhronizovan svaka dva minuta, njegova poslovna starost može biti šest meseci. Zato `maxAge` ne računaš iz `refreshTime` i ne nazivaš ga „password age“. Razdvoji svežinu sinhronizacije, validaciju trenutnog sadržaja i pouzdanu evidenciju rotacije.

10.5 Primer povezivanja

```yaml
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: payments-source
  namespace: payments
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: team-secret-store
    kind: SecretStore
  target:
    name: payments-api-secrets
    creationPolicy: Owner
  dataFrom:
    - extract:
        key: production/payments-api
---
apiVersion: secrets.codepop.tech/v1alpha1
kind: SecretContract
metadata:
  name: payments-api
  namespace: payments
spec:
  secretRef:
    name: payments-api-secrets
  requiredKeys:
    - name: DB_PASSWORD
      minLength: 16
  externalSecretRef:
    name: payments-source
    apiVersion: external-secrets.io/v1
  policy:
    requireExternalSecretReady: true
```

Primer pretpostavlja da je `team-secret-store` već bezbedno konfigurisan. Ne sadrži cloud credential i ne propisuje novi provider authentication model. Secret Contract Operator-u nije potrebna dozvola za čitanje samih SecretStore konfiguracija da bi proverio aplikacioni ugovor.

10.6 Kompatibilnost i testiranje

Za svaku zvanično podržanu ESO verziju testiraj bar: target eksplicitno zadat, target default, Ready True/False, odsutan condition, neispravan tip uslova, CRD odsutan, objekat obrisan i promenjen target. Ako API version nije podržan, prikaži jasan razlog umesto tihog fallback-a.

Envtest može instalirati minimalni test CRD i testirati adapter, ali to nije zamena za kind test sa stvarnim podržanim ESO izdanjem. Lažna šema često propusti razlike u default vrednostima i status ponašanju.

> **Kontrolna tačka.** Ukloni ESO iz test klastera. Ugovor bez ESO reference i dalje treba da radi; ugovor sa obaveznom ESO proverom treba jasno da prijavi nedostupnu integraciju.

11. Provera potrošača i env konfiguracije

11.1 Referenca nije isto što i efektivna promenljiva

Deployment može sadržati `envFrom` referencu na pravi Secret, ali zatim eksplicitni `env` unosi mogu promeniti istu promenljivu. Postojanje reference nije dovoljno da se tvrdi da aplikacija dobija vrednost koju ugovor proverava.

Kubernetes podržava eksplicitne i grupne izvore environment promenljivih; pravila prioriteta izvora moraju biti uzeta u obzir pri proveri potrošnje. Za ovaj projekat konzervativna politika je da konfliktna ili neodrediva konfiguracija ne dobije pozitivan workload nalaz. S16

11.2 Izbor kontejnera

Svaki imenovani potrošač mora postojati. Ako je naveden `containers: [api]`, healthy `metrics` sidecar ne može zadovoljiti ugovor umesto `api` kontejnera. Kada je izabrano više kontejnera, svaki mora zadovoljiti pravilo; ne koristi logiku „dovoljno je da jedan koristi tajnu“.

Init kontejneri podrazumevano nisu deo cilja. Njihova provera mora biti eksplicitna, a mutation nad njima treba da ostane isključen dok nije posebno podržan. Tajna koja treba aplikaciji ne mora biti primerena migracionom ili servisnom init kontejneru.

11.3 Režim EnvFrom

U ovom režimu mora postojati `envFrom.secretRef.name` koji pokazuje na ciljni Secret. Za obaveznu potrošnju `optional: true` ne zadovoljava strogi ugovor. Prefix mora biti odsutan ili eksplicitno podržan API-jem; ovaj početni API ne dodaje implicitnu podršku za prefix.

Ako se ključ `DB_PASSWORD` eksplicitno definiše u `env`, proveri izvor. Ako dolazi iz drugog Secret-a ili inline vrednosti, prijavi konflikt. Ako dodatni kasniji `envFrom` izvor može prepisati ključeve, a operator ne proverava njegov sadržaj, prijavi `AmbiguousEnvPrecedence` umesto nagađanja.

Ne čitaj proizvoljne dodatne Secret-e samo da bi razrešio prioritet. Time bi workload validacija neprimetno proširila osetljivi opseg operatora. Prednost ovog proizvoda je predvidivost, a ne podudaranje sa svakom mogućom složenom konfiguracijom.

11.4 Režim EnvVars

Za svaki obavezni ključ potrebno je odgovarajuće env ime: `envName`, ako postoji, ili naziv ključa. Referenca mora imati tačan Secret naziv i tačan key. `optional` za obavezno pravilo mora biti odsutan ili `false`.

```
env:
  - name: PROVIDER_CONFIG
    valueFrom:
      secretKeyRef:
        name: payments-api-secrets
        key: provider.json
        optional: false
```

Ugovor ne određuje vrednost promenljive kroz `value`. On samo proverava referencu. Opciono pravilo ne zahteva potrošnju odsutnog ključa; ako se kasnije uvede managed injection, njegov `secretKeyRef.optional` mora pratiti optional semantiku.

11.5 Režim Any

`Any` prihvata jedno od podržanih, nedvosmislenih povezivanja. Ne znači da je dovoljno bilo kakvo pominjanje Secret imena negde u PodSpec-u. Volume mount, argument procesa ili annotation nisu automatski env potrošnja.

Volume i CSI modeli mogu postati zasebne potrošačke strategije u budućnosti. U MVP-u status treba da kaže `UnsupportedConsumptionMode` za nepodržanu konfiguraciju, ne da tvrdi da je sadržaj fajla proveren.

11.6 Workload status

Za svaki workload čuvaj identitet, listu proverenih kontejnera, `configured` i stabilan razlog. Korisni razlozi su `WorkloadNotFound`, `ContainerNotFound`, `RequiredReferenceMissing`, `OptionalReferenceNotAllowed`, `EnvConflict` i `AmbiguousEnvPrecedence`.

Prikaz sme sadržati naziv env promenljive i ključ, ali nikad postojeću inline vrednost koja je izazvala konflikt. Konflikt se dijagnostikuje putanjom, na primer `containers[api].env[DB_PASSWORD]`, ne prikazivanjem sadržaja.

11.7 Zamka prvog deploy-a

Ako je `requireWorkloadReference: true`, a workload još ne postoji, `Ready` ne može postati True. Pipeline koji prvo čeka takav Ready i tek onda kreira workload pravi kružnu zavisnost.

Rešenje je podela faza: pre prvog workload deploy-a proveri samo Secret ugovor ili napravi zaseban preflight ugovor bez obavezne workload reference. Posle primene workload-a proveri potrošnju i aplikacioni rollout. Ne prikrivaj ovaj problem time što nepostojeći workload privremeno označiš kao ispravan.

> **Kontrolna tačka.** Dodaj namerni inline `DB_PASSWORD` konflikt u test Deployment. Operator mora prijaviti konflikt bez otkrivanja te inline vrednosti.

12. Opcioni injection i bezbedna izmena workload-a

12.1 Zašto ovo nije deo početnog profila

Dodavanje Secret reference u workload može omogućiti tom workload-u pristup tajni koju ranije nije imao. Operator sa workload `patch` dozvolom postaje privilegovani posrednik. Zato injection nije samo pogodnost za developera već bezbednosna odluka.

Predloženi `v0.2` prvo podržava eksplicitno autorizovane Deployment-e. StatefulSet i DaemonSet dolaze tek uz proveru strategije i odgovarajuće e2e testove. API može unapred ograničiti podržane kind-ove, ali release beleške moraju jasno navesti koji mutation putevi su zaista podržani.

12.2 Obavezne preprovere

Mutation je dozvoljen samo kada instalacija ima uključenu funkciju i write RBAC, ugovor je autorizovan, Secret i sva relevantna konfigurisana bezbednosna pravila su ispravni, ciljni workload je odobren i imenovani kontejneri postoje. Konflikt jednog cilja ne sme dovesti do nedokumentovanog delimičnog menjanja drugih ciljeva.

Preporuka je planiranje svih promena pre prvog upisa, uz status za svaki cilj. Kubernetes ne daje atomsku transakciju preko više workload-a: ako treći patch ne uspe posle prva dva, status mora pošteno prikazati delimičan rezultat i sledeći reconcile mora nastaviti idempotentno.

12.3 Režimi i vlasništvo polja

`injection.mode: Disabled` je default. `EnvFrom` dodaje jednu referencu bez promene postojećeg redosleda i bez duplikata. `EnvVars` dodaje samo nedostajuća eksplicitna mapiranja. Postojeće env ime sa drugim izvorom je konflikt; ne prepisuje se.

Ne dodaj stvarne vrednosti u `env.value`. Ne upravljaj image-om, komandama, service account-om, volumena ili sidecar-ima. Ne uklanjaj korisničku konfiguraciju samo zato što je nema u ugovoru. Usko vlasništvo polja treba dokumentovati pre prvog implementiranog patch-a.

12.4 Patch bez izgubljenih promena

Merge patch nad listom kontejnera može nenamerno prepisati konkurentnu izmenu ako radi sa zastarelim objektom. Koristi read-modify-patch sa optimističkom proverom verzije ili dobro definisan server-side apply model sa uskim vlasništvom. Ne koristi `force` kao podrazumevani izlaz iz ownership konflikta.

Po konfliktu ponovo učitaj workload, proveri odobrenje, ponovo napravi plan i tek onda pokušaj upis. U suprotnom novi sidecar, env unos ili securityContext koji je dodao drugi kontroler može nestati.

12.5 GitOps vlasništvo

Ako GitOps i operator uporno menjaju isto polje, dobićeš drift petlju i ponovljene rollout-e. Najjednostavniji production default ostaje: Git/Helm renderuje env reference, operator ih proverava.

Za managed injection dokumentuj tačne putanje koje pripadaju operatoru i konfiguraciju konkretnog GitOps alata. Ne ignoriši ceo `spec.template` u diff-u. To bi moglo sakriti bezbednosno važnu izmenu image-a ili service account-a.

12.6 Zaštićeno odobrenje

Primer koncepta odobrenja je annotation na samom workload objektu koja sadrži UID odobrenog ugovora. Ta annotation mora biti zaštićena admission ili administrativnim pravilom od neovlašćenih izmena. Sam string nije dovoljna zaštita.

```
metadata:
  annotations:
    secrets.codepop.tech/approved-contract-uid: >-
      317d81e8-2222-4444-8888-7491f91aa744
```

Ovo je projektovani mehanizam, ne već implementirana Kubernetes funkcija. Napredniji sistem može imati zaseban grant resurs sa kontrolisanim autorstvom. U oba slučaja se Secret i workload odobrenje proveravaju pri svakoj promeni, ne samo kada je ugovor prvi put kreiran.

12.7 Idempotentnost i gašenje funkcije

Drugi reconcile ne dodaje novu istu referencu. Isključivanje mutation funkcije odmah prekida nove promene, ali ne uklanja automatski već dodate env reference. To je konzervativna politika očuvanja dostupnosti; uklanjanje zahteva eksplicitnu administrativnu proceduru.

Ne pokušavaj automatski revert ako se workload u međuvremenu promenio. Stari snapshot nije autorizacija da se prepiše novo stanje. Operator treba da objavi šta je uradio i šta je prestao da radi, a ne da glumi opšti konfiguracioni backup sistem.

> **Kontrolna tačka.** Testiraj dva istovremena patch-a: jedan dodaje env referencu kroz operator, drugi dodaje legitimnu promenu kroz korisnika. Nijedna promena ne sme biti tiho izgubljena.

13. Rotacija i kontrolisani rollout

13.1 Tri različita događaja

Promena objekta, uspešna sinhronizacija i rotacija kredencijala nisu isto. Label update menja Kubernetes objekat bez promene lozinke. ESO može potvrditi isti sadržaj bez poslovne rotacije. Provider može promeniti lozinku pre nego što novi sadržaj stigne do klastera.

Operator zato odvojeno beleži opaženi identitet Secret-a, stanje deklarisane rotacione politike i eventualno zatraženu workload promenu. Ne naziva svaku novu `resourceVersion` vrednost „novom lozinkom“.

13.2 Env vrednosti aktivnog procesa

Kontejner koji koristi Secret kao environment promenljive ne dobija automatski novu vrednost samo zato što se Secret promenio. Potrebno je ponovno pokretanje odgovarajućeg procesa odnosno podova. Kubernetes dokumentacija tu razliku eksplicitno navodi. S17

Predloženi restart mehanizam menja PodTemplate annotation. Time odgovarajući workload kontroler dobija novi template. Secret Contract Operator ne briše podove direktno i ne komanduje procesu da ponovo učita konfiguraciju.

13.3 Token bez sadržaja tajne

```
annotation key:
  secrets.codepop.tech/restart-<contract-uid>

annotation value:
  <secret-uid>:<secret-resource-version>
```

UID ugovora je metadata identitet i razlikuje ponovno kreiran ugovor. UID Secret-a razlikuje novi objekat sa starim imenom. `resourceVersion` se koristi samo kao identifikator opažene verzije; token ne uključuje hash podataka.

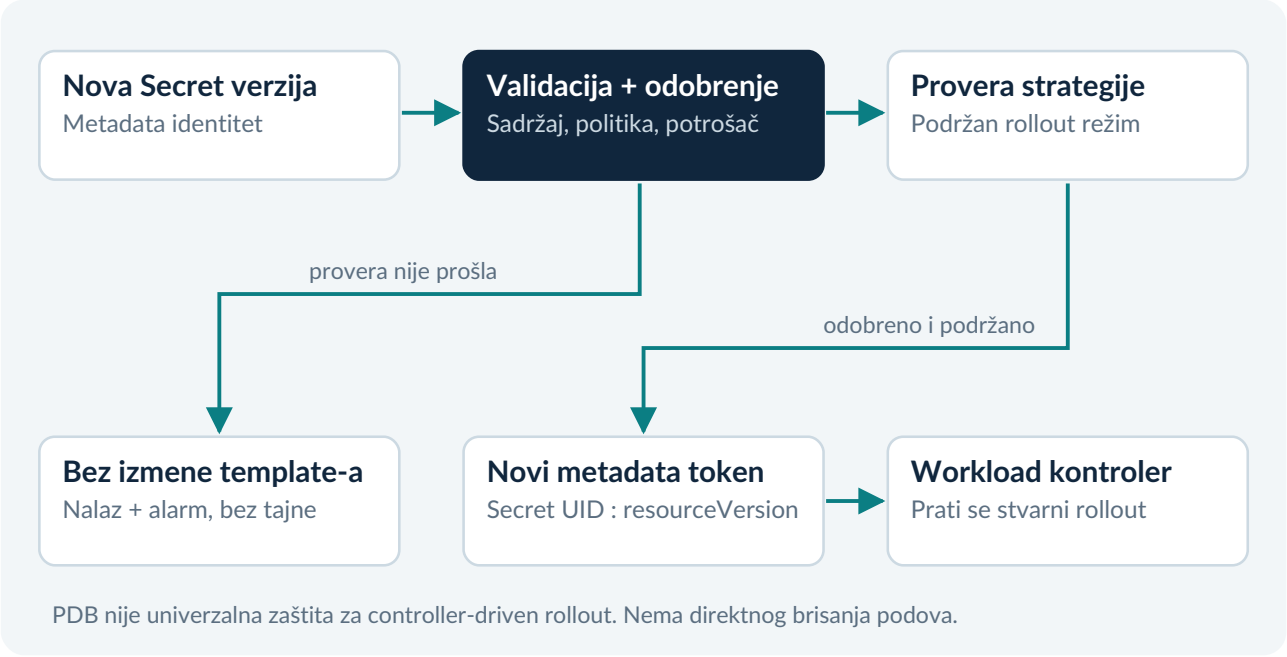
Podrazumevana politika ove knjige je **restart na odobrenu promenu Secret objekta**, ne precizno „samo na promenu vrednosti“. I metadata update može izazvati rollout. To ograničenje mora biti u README-ju, jer se inače ponašanje doživljava kao greška.

Kod prvog uključivanja funkcije postavljanje annotation-a samo po sebi menja template i može izazvati inicijalni rollout. Ovo izdanje bira eksplicitni inicijalni rollout, umesto skrivenog baseline stanja. Korisnik uključuje opciju u planiranom prozoru.

13.4 Ne restartuj zbog neispravne nove tajne

Ako novi Secret ne zadovoljava obavezne ključeve ili bezbednosnu politiku, postojeći podovi ne treba automatski da se zamene neispravnim. Operator prijavljuje nalaz i ne menja restart annotation.

To ipak nije puna zaštita: pod koji se nezavisno rekreira zbog kvara noda može pročitati trenutno neispravan Secret. Zato zadržavanje starog aktivnog procesa nije isto što i bezbedan model rotacije. Za stroži model koristi verzionisane, nepromenljive Secret objekte i kontrolisano prebacivanje reference nakon validacije.



Slika 13.1 — Bezbednosne provere pre signala za rollout; neuspešna validacija ne menja template.

13.5 Strategije workload-a

Deployment rollout treba da proveri pauzu i update strategiju. `RollingUpdate` ima parametre dostupnosti koje vlasnik aplikacije podešava; `Recreate` ima drugačiju dostupnosnu semantiku. Operator ne treba neprimetno da menja strategiju zbog rotacije. S18

StatefulSet sa `OnDelete` ne zamenjuje automatski podove nakon template promene. `RollingUpdate` može imati partition koji ograničava koji podovi se ažuriraju. Za početni restart profil zahtevaj podržan RollingUpdate oblik bez neobrađenih ograničenja; ostalo prijavi kao `UnsupportedRolloutStrategy`. S19

DaemonSet takođe zahteva razumevanje svog update režima. `OnDelete` ne sme biti predstavljen kao uspešan automatski restart. Podrška za ovaj kind prihvata se tek kada e2e test proveri stvarnu zamenu podova u izabranoj strategiji. S20

PodDisruptionBudget nije univerzalna zaštita za ovu operaciju. Rolling update kojim upravlja Deployment ili StatefulSet nije ograničen PDB-om na isti način kao Eviction API. Dostupnost se podešava kroz konkretnu workload strategiju, readiness i kapacitet klastera. S21

13.6 Politika maksimalne starosti

`maxAge: 720h` označava 30 perioda po 24 sata. Operator čita odobrenu annotation vrednost kao RFC3339 timestamp i poredi je sa ubrizganim satom. Nedostajuća annotation je `RotationTimestampMissing`, neispravan datum `RotationTimestampInvalid`, budući datum van tolerancije `RotationTimestampInFuture`, a istekao rok `SecretTooOld`.

Ta provera potvrđuje samo tvrdnju pouzdanog pisca metapodatka. Ako korisnik može proizvoljno promeniti timestamp, može resetovati deklarisanu starost bez rotacije. Prava kontrola zahteva dokumentovano poreklo i ovlašćenje za upis tog metapodatka.

13.7 Storm kontrola

Brze promene Secret-a mogu napraviti više rollout zahteva. Dodaj dokumentovan minimum između restart zahteva i kratko debounce objedinjavanje, ali ne izgubi najnoviju odobrenu verziju. Pending opažanje treba ponovo proveriti pre upisa.

Dok traje rollout, promena nove verzije ne znači da treba odmah emitovati deset novih patch-eva. Prva implementacija može dozvoliti samo jedan aktivni ciklus po workload-u, a zatim proveriti poslednju opaženu verziju. Ovakva politika je dodatna state machine i mora imati testove za restart operatora.

13.8 Rollback granica

Operator nikada ne vraća staru tajnu iz sopstvene memorije ili statusa, jer je tamo ne čuva. Rollback kredencijala pripada pouzdanom izvoru i njegovoj proceduri. Rollback aplikacije ne vraća automatski prethodnu lozinku, a vraćanje lozinke ne garantuje da je provider ponovo prihvata.

> **Kontrolna tačka.** Rotaciona demonstracija mora uključiti neuspešnu novu vrednost, metadata-only promenu, ponovno kreiranje Secret-a i nekompatibilnu rollout strategiju.

14. CI, GitOps i stvarna kontrola deploy-a

14.1 Status nije enforcement

Kubernetes neće odbiti Deployment samo zato što neki `SecretContract` ima `Ready=False`. Veza između njih postoji samo ako je napravi pipeline, GitOps konfiguracija ili poseban admission sloj. Status je signal, ne automatsko izvršno pravilo.

Minimalni CI obrazac glasi: primeni ugovor za Secret, sačekaj rezultat za očekivanu generaciju, primeni workload, sačekaj workload potrošnju i proveri aplikacioni rollout. Nema čitanja Secret vrednosti u pipeline-u.

14.2 Problem zastarelog Ready

Običan `kubectl wait --for=condition=Ready` može zateći pozitivan condition iz prethodne generacije. Zato gate proverava da `status.observedGeneration` i `Ready.observedGeneration` odgovaraju očekivanoj generaciji i da UID ugovora nije promenjen.

Čak i kada te provere prođu, Secret se može promeniti pre kreiranja poda. To je vremenska trka između provere i upotrebe. Nije pošteno takav pipeline nazvati atomskim ili race-free deploy-om.

14.3 Referentna gate skripta

Ova skripta proverava identitet i generaciju ugovora, ne čita Secret. Pripada budućem repozitorijumu koji ima implementiran kanonski status. Namerno ne prikazuje ceo objekat pri grešci.

```
#!/usr/bin/env bash
set -euo pipefail

NS="${NS:-payments}"
CONTRACT="${CONTRACT:-payments-preflight}"
TIMEOUT_SECONDS="${TIMEOUT_SECONDS:-120}"
[[ "$TIMEOUT_SECONDS" =~ ^[1-9][0-9]*$ ]] || exit 2

initial="$(kubectl -n "$NS" get scn "$CONTRACT" -o json)"
uid="$(jq -er '.metadata.uid' <<<"$initial")"
gen="$(jq -er '.metadata.generation' <<<"$initial")"
deadline=$((SECONDS + TIMEOUT_SECONDS))

while (( SECONDS < deadline )); do
  if object="$(kubectl -n "$NS" get scn "$CONTRACT" -o json)"; then
    if jq -e --arg uid "$uid" --argjson gen "$gen" '
      .metadata.uid == $uid and
      .metadata.generation == $gen and
      .status.observedGeneration == $gen and
      (.status.observedSecret.uid | length) > 0 and
      any(.status.conditions[]?;
```

```
        .type == "Ready" and
        .status == "True" and
        .observedGeneration == $gen)
    ' <<<"$object" >/dev/null; then
      echo "Secret contract accepted for expected generation."
      exit 0
    fi
  fi
  sleep 2
done

echo "Secret contract gate timed out." >&2
exit 1
```

Ako ugovor za vreme čekanja dobije novu generaciju, skripta ne prihvata rezultat nove konfiguracije kao da pripada prvobitnom zahtevu. Tim može dopuniti skriptu ranijim prekidom i eksplicitnom porukom o promenjenom identitetu.

Za strožu svežinu potreban je projektovan protokol ručno zatražene provere sa potvrđenim request ID-jem u statusu ili specifično vezivanje za verzionisani Secret. Ova skripta takvu garanciju nema i ne treba je predstavljati kao da je ima.

14.4 Dve faze prvog puštanja

Preflight ugovor nema obaveznu workload referencu. On proverava sadržaj i eventualnu ESO zavisnost. Posle njegove potvrde primenjuje se Deployment sa Git-owned env referencama. Drugi ugovor ili post-deploy faza proverava da potrošači koriste pravi Secret.

Takva podela uklanja kružnu zavisnost u kojoj ugovor čeka Deployment koji pipeline još ne dozvoljava. Pri sledećim izdanjima postojeći workload ne menja činjenicu da status potvrđuje samo provereni PodTemplate, ne buduću Git izmenu koja još nije primenjena.

14.5 GitOps model

Konkretan GitOps alat mora znati kako tumači health custom resursa i zavisnosti sinhronizacije. Običan redosled primene fajlova nije dovoljan dokaz da je prethodni resurs postao spreman. Implementacija integracije se radi prema zvaničnoj dokumentaciji izabranog alata i testira u namenskom klasteru.

Knjiga bira Git-owned env reference kao default. Managed injection se uključuje samo u instalaciji koja ima jasan dogovor o vlasništvu. Cilj je izbeći da GitOps briše annotation, operator je vraća, a aplikacija se stalno ponovo podiže.

14.6 Sinhrona admission zabrana

Poseban validacioni webhook mogao bi odbiti određeni workload upis. Ali webhook postaje kritična zavisnost: potrebno je rešiti timeout, TLS, dostupnost, failure policy, opseg namespace-ova, dry-run ponašanje i break-glass proceduru. Kubernetes smernice preporučuju usko ciljane i pouzdane admission kontrole i razmatranje ugrađenih alternativa kada odgovaraju. S22

Webhook koji proverava samo zastareli `Ready` nije automatski bezbedan. Webhook koji čita Secret uživo smanjuje jednu trku, ali i dalje nema atomsku garanciju da će pod kasnije pročitati istu promenljivu vrednost. Za stroži model razmotri immutable, verzionisane Secret reference i autorizovano vezivanje ugovora za tačan cilj.

14.7 Pravila koja ne treba obećati

Nemoj navoditi da CRD schema validacija može proizvoljno čitati drugi Secret. Nemoj tvrditi da `Ready` čuva aplikaciju od svake kasnije neispravne rotacije. Nemoj `fail-open` režim nazvati tvrdom zabranom niti `fail-closed` režim uključiti za ceo klaster bez recovery plana.

> **Kontrolna tačka.** Nacrtaj vremensku liniju između validacije, izmene workload-a i čitanja env-a pri startu procesa. Svaki razmak predstavlja granicu garancije koju dokumentacija mora priznati.

15. Strategija testiranja i dokazivanje invarijanti

15.1 Test piramida ovog projekta

Najveći broj testova pripada čistom validatoru i status builder-u. Envtest proverava stvarni API, default vrednosti, status subresource i event-driven reconcile. Kind testovi proveravaju instalaciju, RBAC, stvarni workload rollout i integraciju sa podržanim ESO izdanjem.

Ove grupe imaju različite ciljeve. Green unit suite nije dokaz da Helm chart daje ispravne dozvole. Green kind demonstracija nije dokaz da validator ne odaje podatke u neobičnoj parser grešci.

15.2 Matrica validatora

Oblast	Pozitivan slučaj	Negativan slučaj
Obaveznost	Ključ postoji	Ključ odsutan
Optional	Odsutan optional se preskače	Prisutan optional je neispravan
Praznina	Jedan ili više bajtova	Nula bajtova
Dužina	Granice uključene	Ispod minimuma / iznad maksimuma
UTF-8	Jasna byte semantika	Pogrešna pretpostavka o znakovima
Regexp	Sidra i substring slučajevi	Invalid pattern / mismatch
JSON	Objekat i skalar	Oštećen JSON
URI	Apsolutni mrežni i opaque URI	Relativni URI
PEM	Jedan blok i bundle	Smeće pre ili posle bloka
Budžet	Granična veličina	Prevelik ulaz

Svaki slučaj ima jasan očekivani reason. Provere ne smeju zavisiti od redosleda Go map iteracije. Poredi sortirane nalaze i stabilnu JSON reprezentaciju rezultata.

15.3 Envtest matrica

Osnovni scenario kreira ugovor pre Secret-a i očekuje `Ready=False`. Kreiranje Secret-a mora preko watch-a promeniti status bez ručnog poziva reconcile metode. Zatim ukloni ključ, vrati ga i obriši Secret. Svaki prelaz treba proveriti.

Posebno testiraj generaciju, no-op upise, dva ugovora sa istim Secret imenom u različitim namespace-ovima, restart manager-a i promenu odobrenja. Envtest nema standardne workload kontrolere, pa test proverava PodTemplate patch, ne stvarnu dostupnost podova. S10

Konflikt status upisa mora pokrenuti ponovno čitanje. Test za ovaj slučaj treba namerno promeniti CR između prvog opažanja i patch-a. Nije dovoljno da fake client uvek prihvati promenu.

15.4 Testovi bezbednosnog izlaza

Koristi sentinel koji se razlikuje od naziva ključa i pravila. Uhvatiti treba status JSON, event poruke, log zapis, annotation promene i metrics exposition. Proveri da nema sirove, base64 i hex reprezentacije sentinel-a, kao ni njegovih uobičajenih deterministčkih digest-a.

Ovaj test nije matematički dokaz da svaki deo vrednosti nikada ne može procureti. Zato se kombinuje sa dizajnom rezultata bez polja za ulaz i pregledom svih formatiranja grešaka. Posebno testiraj neispravan URI i PEM, jer parser greška može drugačije formatirati payload.

Ne koristi realne tajne u fixture-ima. Test neuspeh ne treba da ispiše ceo ulaz ili ceo Kubernetes Secret objekat. Čak i sintetički primeri treba da modeluju bezbedno rukovanje.

15.5 Negativni RBAC testovi

Podigni manager sa namespaced Role-om. Potvrdi da ne može da čita Secret iz drugog namespace-a, da ne može da pravi ili menja Secret i da validation-only profil ne može da patch-uje workload.

Koristi eksplicitni ServiceAccount identitet. Test iz administratorskog kubeconfig-a koji uspešno menja workload ne dokazuje ništa o operatorovim dozvolama. Za mutation profil proveri i negativan slučaj bez odobrenja na cilju.

15.6 E2E scenariji

Stvarni klaster treba da potvrdi: chart installation, readiness manager-a, CRD default, uspešan ugovor, neuspešan ugovor, izmenu Secret-a, restart operatora, cleanup i odsustvo neželjenog brisanja aplikacionih resursa.

Za opcioni restart testiraj Deployment RollingUpdate, pauzirani Deployment, StatefulSet OnDelete, podržan StatefulSet RollingUpdate i DaemonSet režime. Broj novih podova i završni rollout status proveravaju se kroz metadata i workload status, bez `kubectl exec env`.

15.7 Chaos i degradacija

Prekini pristup API-ju u kontrolisanom testu, opozovi jednu RBAC dozvolu, ukloni ESO CRD, pošalji više brzih Secret promena i ugasi aktivnog leader-a. Operator ne sme panično pisati podatke u log, nekontrolisano restartovati aplikaciju ili trajno izgubiti najnoviji nalaz.

Test clock-a koristi ubrizgano vreme, ne višeminutne `sleep` pozive. Testiraj granicu isteka, dozvoljeni skew, daleku budućnost i restart kontrolera neposredno pre roka.

15.8 Kriterijumi prihvatanja

Svaki PR mora navesti koje testove je izvršio i koji nisu mogli da se izvrše. CI artefakt sadrži izlaz testova bez payload-a tajni. Coverage procenat je pomoćni signal; ne zamena za pokrivanje autorizacije, status svežine i idempotentnosti.

Predloženi release minimum je: unit i race provere, envtest za svaku podržanu Kubernetes minor verziju koju release oglašava, chart render provere i bar jedan kind scenario po aktivnoj integraciji. Ovo su kriterijumi koje projekat tek treba da ispuni, a ne već ostvarena matrica.

> **Kontrolna tačka.** Za svaku javnu garanciju pronađi bar jedan test koji bi pao kada bi se garancija prekršila. Garancija bez takvog testa zahteva dodatni review.

16. Metrike, događaji i operativni signal

16.1 Tri odvojena pitanja

Da li je operator zdrav? Da li ugovori zadovoljavaju pravila? Da li se aplikacije uspešno izvršavaju? To su tri različite klase signala. `Ready=False` za jedan ugovor ne treba da sruši liveness manager-a, a healthy manager ne znači da su svi ugovori zeleni.

Liveness proverava da proces može da nastavi rad. Readiness manager-a proverava operativne preduslove kao što su pokretanje kontrolera i potrebna inicijalizacija. Metrike ugovora opisuju domenski rezultat.

16.2 Projektovane metrike

Metrika	Tip	Značenje
`secret_contract_ready`	Gauge	1 samo za potvrđen trenutni Ready
`secret_contract_missing_keys`	Gauge	Broj nedostajućih obaveznih ključeva
`secret_contract_violations`	Gauge	Broj objavljenih ili ukupnih nalaza, jasno definisati
`secret_contract_checks_total`	Counter	Broj evaluacija po ograničenom ishodu
`secret_contract_check_duration_seconds`	Histogram	Trajanje evaluacije
`secret_contract_mutations_total`	Counter	Odobrene mutation operacije po ishodu

Prometheus preporučuje stabilno imenovanje, jasne jedinice i kontrolu kardinalnosti label-a. Svaki različit skup label vrednosti pravi novu vremensku seriju. S23

`namespace` i ime ugovora mogu biti prihvatljivi za opt-in per-object metrike, ali i oni mogu postati skupi. Globalni brojači koriste mali enum ishoda. Ne dodaj UID, resourceVersion, naziv svakog Secret ključa, pattern, poruku greške ili vrednost kao label.

16.3 Unknown i nekonfigurisane provere

Za glavni `ready` gauge 1 znači potvrđeno True, a 0 sve ostalo. Dodatna condition metrika može eksplicitno razlikovati True, False i Unknown kroz ograničen enum. Ne koristi NaN kao neobjašnjiv signal u osnovnom dashboard-u.

Nepodešena opciona integracija nije greška. Dashboard treba da prikaže `NotConfigured` ili da izostavi relevantnu per-integration seriju, u skladu sa dokumentovanim pravilom. Bitno je da alarm za ESO ne okida ugovore koji ESO ne koriste.

16.4 Čišćenje serija

Kada se ugovor obriše, ukloni njegove gauge serije iz collector-a. U suprotnom će nepostojeći ugovor ostati crven ili zelen sve dok se operator ne restartuje. Ako postoji više replika, način izlaganja per-object metrika mora sprečiti zabunu između leader i standby pogleda.

Counters po objektu zahtevaju dodatnu pažnju zbog churn-a; bolje je koristiti agregirane counters i ograničene per-object gauge metrike. Testiraj kreiranje i brisanje velikog broja kratkotrajnih ugovora.

16.5 Događaji

Event `ContractReady` emituj pri relevantnom prelazu. `SecretValidationFailed` emituj pri novom skupu nalaza, ne na svakom identičnom reconcile-u. `WorkloadPatched` označava uspešan API patch, ne završen rollout.

Poruke su lokalno generisane, kratke i bez sirovih parser ili provider grešaka. Primer: „Required key DB_PASSWORD is missing.“ U strožem metadata režimu poruka može biti „One or more required keys are missing.“

16.6 Zaštita metrics endpoint-a

Metrike mogu otkriti nazive aplikacija, namespace-ove i stanje internih sistema. Endpoint ne treba javno izlagati bez kontrole. Instalacija definiše TLS i autentikaciju ili drugi dokumentovan privatni transportni model, uz NetworkPolicy gde mrežna implementacija to podržava.

ServiceMonitor je opcion resurs koji postoji samo kada odgovarajući Prometheus Operator CRD postoji. Njegovo uključivanje ne sme srušiti podrazumevanu instalaciju klastera bez Prometheus Operator-a.

16.7 Predloženi alarmi

Alarm za ugovor koji je neispravan duže od pet minuta može biti početna operativna politika, ne univerzalni standard. Alarm za visoku reconcile error stopu pripada održavaocu operatora. Alarm za aplikacioni pad pripada vlasniku aplikacije. Nemoj ista upozorenja slati svima.

Posebno prati dugotrajno odsustvo novih opažanja kada postoje očekivane promene, učestale API konflikte, metadata-only restart oluje i stalnu nedostupnost ESO adaptera. Nazivi razloga treba da ostanu stabilni između patch izdanja.

> **Kontrolna tačka.** Iz metrics endpoint-a ne sme biti moguće dobiti sadržaj tajne, njenu dužinu po ključu ili deterministički otisak. Proveri i nepotrebnu kardinalnost.

17. Pakovanje, RBAC i instalacija

17.1 Dva instalaciona profila

Podrazumevani profil je namespace-scoped i validation-only. Kontroler čita odobreni opseg, menja status ugovora i objavljuje događaje. Opciono čita ESO i workload resurse. Nema pravo da kreira, menja ili briše Secret-e i nema workload write dozvolu.

Mutation profil je zasebno uključenje koje dodaje minimalni workload `patch` i potrebnu implementaciju autorizacije. Helm `values.yaml` ne treba da podrazumevano instalira snažniji ClusterRole samo zato što buduća funkcija postoji u binarnom fajlu.

17.2 Namespaced reader Role

Sledeći manifest je referentni RBAC isečak za posmatrani aplikacioni namespace. Leader election i manager namespace dozvole definišu se odvojeno. ESO pravilo uključuje se samo kada je integracija aktivna.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: secret-contract-reader
  namespace: payments
rules:
  - apiGroups: [secrets.codepop.tech]
    resources: [secretcontracts]
    verbs: [get, list, watch]
  - apiGroups: [secrets.codepop.tech]
    resources: [secretcontracts/status]
    verbs: [get, patch, update]
  - apiGroups: [""]
    resources: [secrets]
    verbs: [get, list, watch]
  - apiGroups: [apps]
    resources: [deployments, statefulsets, daemonsets]
    verbs: [get, list, watch]
  - apiGroups: [""]
    resources: [events]
    verbs: [create, patch]
```

Ovaj Role dopušta čitanje svih Secret-a u namespace-u. Nije per-Secret izolacija. U striktnijem profilu moraš uskladiti stvarne `get` dozvole, watch model i odobrene reference. `resourceNames` nije čarobno rešenje za proizvoljni informer koji lista sve objekte.

17.3 Identitet i leader election

Manager dobija namenski ServiceAccount, bez cluster-admin prava. RoleBinding u aplikacionom namespace-u može pokazivati na ServiceAccount iz operator namespace-a. Lease dozvole za leader election ograniči na manager namespace i konkretne potrebne resurse.

Dve replike sa leader election-om povećavaju dostupnost kontrolne komponente, ali ne udvostručavaju automatski kapacitet reconcile-a. Standby proces i dalje može imati određene cache i metrics karakteristike koje treba proveriti u izabranoj konfiguraciji.

17.4 Pod hardening

```
securityContext:
  runAsNonRoot: true
  seccompProfile:
    type: RuntimeDefault
containers:
  - name: manager
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: [ALL]
    resources:
      requests:
        cpu: 100m
        memory: 128Mi
      limits:
        memory: 256Mi
```

Ovo su početne vrednosti za testiranje, ne izmerena potreba operatora. Read-only filesystem može zahtevati eksplicitni privremeni volume za određene funkcije; ne isključuj hardening bez objašnjenja. Image ne treba da sadrži shell ako runtime-u nije potreban.

NetworkPolicy treba da dozvoli potrebnu API komunikaciju i metrics scrape, uz DNS samo gde je zaista potreban. Tačan API endpoint i ponašanje mrežne implementacije zavise od klastera; generički manifest ne treba obećavati kao univerzalno primenljiv firewall.

17.5 Helm chart ugovor

Chart treba da omogući image digest, resurse, watch namespace listu, securityContext, ServiceAccount, reader RBAC, opcioni mutation RBAC, metrics i opcioni ServiceMonitor. Image tag ne sme ostati implicitno `latest` u production primeru.

Default chart ne zahteva cert-manager ako nema webhook-a. Ako se webhook kasnije uvede, dokumentuj da li TLS isporučuje cert-manager ili drugi podržani mehanizam. Cert-manager nije inherentni zahtev svakog operatora.

17.6 CRD životni ciklus

Helm CRD fajlovi u `crds/` imaju poseban životni ciklus: njihova standardna instalacija nije isto što i automatsko upravljanje upgrade-om i brisanjem CRD-a. Zvanična Helm dokumentacija upozorava na ta ograničenja i na granice dry-run provere. S24

Predlog je objaviti CRD manifest kao odvojen release artefakt i dokumentovati redosled: proveri kompatibilnost, primeni novu CRD šemu, zatim upgrade kontrolera i tek potom nove CR funkcije. Ne briši CRD kao deo običnog `helm uninstall` toka.

17.7 Instalacija iz stvarnog release-a

Kada projekat zaista objavi release, README koristi tačnu oznaku i proverljive artefakte. Dok release ne postoji, komande koriste lokalni chart i lokalno izgrađen image. Nemoj navoditi izmišljeni javni registry tag koji korisnik ne može preuzeti.

```
# Preduslov: operator je implementiran i chart postoji.
helm lint charts/secret-contract-operator
helm template sco charts/secret-contract-operator \
  --namespace secret-contract-system > rendered.yaml

# Pregledaj rendered.yaml pre bilo kakvog cluster apply-a.
```

Kontrolna tačka. Renderuj validation-only chart i automatski proveri da nijedno pravilo ne daje `patch`, `update`, `create` ili `delete` nad Secret-ima ili workload-ima, osim jasno dozvoljenog status/event slučaja.

18. CI/CD, release i lanac isporuke

18.1 CI kao proverljivi ugovor

Pull request pipeline treba da proveri formatiranje, statičku analizu, unit testove, race testove, envtest, regeneraciju API artefakata, chart lint i build kontejnera bez push-a. Te provere ne zahtevaju produkcione cloud kredencijale.

Generisani fajlovi su deo izvornog koda. Posle `make generate` i `make manifests`, `git diff --exit-code` treba da bude prazan. Ne popravljaj zastarele CRD-ove tek u release pipeline-u; greška treba da bude vidljiva u PR-u.

18.2 Bezbednost GitHub Actions toka

GitHub preporučuje minimalne token dozvole, pažljivo rukovanje nepouzdanim input-om i pinovanje third-party Actions na pun commit SHA za nepromenljivu referencu. Posebno je rizično izvršavati nepouzdan PR kod sa privilegovanim tokenom. S26

Za obični PR dovoljan je `contents: read` uz dozvole koje test zaista zahteva. Ne push-uj image iz neproverenog fork PR-a. Ne interpoliraj naziv grane ili PR naslov direktno u shell kao kod. Ne koristi `pull_request_target` za checkout i izvršavanje nepoverljivog head koda.

18.3 Predložene pipeline faze

```
pull request
  format + vet
  unit + race
  envtest
  generated-diff check
  chart render + RBAC policy checks
  container build
  optional kind integration

signed/reviewed release tag
  repeat required validation
  multi-arch build
  push immutable digest
  generate SBOM + provenance
  sign and publish artifacts
```

Dijagram opisuje ciljnu organizaciju, ne postojeći workflow. Tačne Actions verzije se biraju i zaključavaju kada se pipeline implementira. Ne ubacuj izmišljene SHA vrednosti u dokumentaciju radi utiska kompletnosti.

18.4 Artefakti release-a

Release sadrži image za podržane arhitekture, CRD manifest, instalacioni manifest ili Helm paket, checksums, SBOM i release beleške. Image digest se navodi uz tag tako da korisnik može zaključati tačan sadržaj.

Potpis artefakta dokazuje poreklo prema izabranom trust modelu, ne ispravnost ponašanja operatora. README treba da pokaže kako korisnik proverava potpis i koji identitet izdavača očekuje. Bez te provere „signed image“ je samo tehnička oznaka.

18.5 Verzije koje treba razlikovati

Application release, chart version i CRD API version nisu isto. `v0.1.3` binarni release može i dalje služiti `secrets.codepop.tech/v1alpha1`. Chart patch može promeniti resurse ili dokumentaciju bez promene CRD šeme.

Ako se promeni semantika `Ready`, default autorizacije ili način izračunavanja restart tokena, to nije bezazlena izmena zato što YAML polja imaju ista imena. Release beleške moraju opisati uticaj na postojeće ugovore i potencijalne rollout-e.

18.6 Testiranje upgrade-a

Prvo instaliraj prethodni stvarni release u kind, kreiraj ugovore i sintetičke Secret-e, pa primeni novu CRD i operator verziju. Proveri da stari ugovori ostaju validni, da status dobija novu očekivanu semantiku i da se ne pokreću neplanirani restarti.

Rollback operator image-a može biti bezbedan samo ako prethodni binary razume trenutne sačuvane CR podatke i šemu. Nemoj obećavati rollback svih API promena jednim `helm rollback` pozivom.

18.7 Šta ne sme u release priloge

Nema kubeconfig-a, punih cluster dump-ova, stvarnih Secret-ova, heap profila ili debug logova sa neproverenim payload-om. Čak i test e2e artefakti prolaze pregled redakcije pre javnog objavljivanja.

Fajl `TEST_REPORT.md` navodi šta je stvarno prošlo, na kojoj verziji i šta nije pokrenuto. „CI planned“ nije isto što i „CI passing“.

> **Kontrolna tačka.** Release treba moći reprodukovati iz čistog, označenog commit-a bez tajnih ručnih koraka na laptopu održavaoca.

19. Operativne procedure i incidenti

19.1 Dijagnostika bez prikazivanja tajne

Prvo pregledaj status ugovora i manager događaje, ne sadržaj Secret-a. Operatorov status treba da bude dovoljan za većinu tipičnih grešaka. Direktno čitanje vrednosti je zasebna, ovlašćena operacija koja se radi samo kada je stvarno potrebna.

```
kubectl -n payments get scn
kubectl -n payments describe scn payments-api
kubectl -n payments get scn payments-api \
  -o jsonpath='{.status.conditions}'

kubectl -n secret-contract-system get deploy,pods
kubectl -n secret-contract-system logs \
  deployment/secret-contract-operator --tail=100
```

Komande pretpostavljaju navedeni installation naziv. Ne koriste `kubectl get secret -o yaml`, `printenv` ili `exec env`. Logovi se prvo pregledaju lokalno i ne prosleđuju javno bez provere.

19.2 Ugovor nije spreman

Razlog	Najverovatniji smer provere	Bezbedna radnja
`SecretNotFound`	Pogrešan naziv ili neuspešan sync	Proveri referencu i ESO status
`MissingRequiredKey`	Promenjen ugovor ili provider struktura	Uporedi nazive ključeva
`InvalidJSON`	Pogrešan format izvora	Vlasnik tajne ispravlja sadržaj
`AccessNotApproved`	Opozvan ili odsutan grant	Proveri administrativno odobrenje
`SecretAccessDenied`	Pogrešan Role/RoleBinding	Proveri operator identitet
`ContainerNotFound`	Promenjeno ime kontejnera	Ispravi workload referencu
`AmbiguousEnvPrecedence`	Više konfliktnih izvora	Pojednostavi env mapiranje

Nemoj „popraviti“ incident globalnim povećanjem prava na cluster-admin. Prvo utvrdi tačno pravo koje nedostaje i da li je uopšte trebalo da postoji. Isto važi za isključivanje obavezne politike da bi status postao zelen.

19.3 Status izgleda zastarelo

Proveri `metadata.generation`, `status.observedGeneration`, condition generacije i identitet opaženog Secret-a. Proveri da li manager radi i da li je izabrani namespace u watch opsegu. Promena Secret-a mora imati put do indeksa i map handler-a.

Ako su samo vremenska pravila zastarela, pregledaj requeue računanje i clock. Ako nova ESO instalacija nije otkrivena, primeni dokumentovanu discovery/restart proceduru. Ne restartuj nasumično sve aplikacione podove da bi „osvežio“ operator status.

19.4 Restart petlja

Privremeno onemogući mutation na nivou instalacije, bez menjanja Secret vrednosti. Zabeleži samo metadata istoriju: koji ugovor je izdao token, za koji Secret UID/RV i koje workload generacije su nastale.

Traži drugi kontroler koji menja istu annotation, GitOps revert, ESO metadata churn ili prekratko debounce pravilo. Kada se razlog ukloni, proveri poslednje odobreno stanje i tek onda ponovo uključi restart funkciju.

19.5 Neispravna rotacija

Ako novi sadržaj nije validan, potvrdi da operator nije izdao novi restart signal. Vlasnik izvora popravlja ili vraća kredencijal po sopstvenom runbook-u. Nakon ispravke proveri sinhronizaciju, ugovor i aplikacioni health kao odvojene faze.

Ne pretpostavljaj da stari podovi sigurno opstaju. Mogu nestati zbog nezavisnog kvara ili skaliranja. Kritične aplikacije treba da imaju verzionisanu rotacionu proceduru i dovoljno dug period preklapanja kredencijala tamo gde izvor podržava takav model.

19.6 Sumnja na curenje

Tretiraj objavljenu vrednost kao potencijalno kompromitovanu. Ograniči pristup log i metrics sistemima, zaustavi rizičnu operator funkciju, sačuvaj redigovanu evidenciju i pokreni ovlašćenu rotaciju pogođenih kredencijala. Brisanje log reda nije zamena za rotaciju.

U tehničkom incident zapisu navedi koji kanal je procurio, koliko dugo je bio dostupan i koje komponente su imale pristup. Ne kopiraj stvarnu tajnu u ticket da bi dokazao da je procurila.

19.7 Uninstall i oporavak

Običan uninstall uklanja kontroler, njegov ServiceAccount i pripadajuće instalacione resurse. Ne treba da briše aplikacione Secret-e ili workload-e. CRD se uklanja samo posebnom odlukom, jer njegovo brisanje utiče na sve instance tog custom resursa.

Pre gašenja mutation profila dokumentuj koje reference i annotation-i ostaju. Naredni vlasnik konfiguracije mora znati šta je Git-owned, a šta je ranije dodao operator. Backup specifikacija ugovora ne predstavlja backup stvarnih kredencijala.

> **Kontrolna tačka.** Operator mora imati postupak gašenja koji ne zahteva čitanje ili vraćanje stvarnih Secret vrednosti iz njegovog sopstvenog stanja.

20. API evolucija i open-source održavanje

20.1 Alpha ne znači proizvoljne promene

`v1alpha1` signalizira da API još nije stabilizovan, ali ne oslobađa održavaoca odgovornosti prema korisnicima. Posle javnog release-a i mala promena default-a može zaustaviti deploy ili pokrenuti rollout. Svaka promena mora imati migracionu napomenu i test starog manifesta.

Kubernetes podržava više served verzija CRD-a i odvojenu storage verziju, uz pravila konverzije i migracije sačuvanih objekata. Samo dodavanje nove verzije u YAML ne migrira automatski sve postojeće podatke. S27

20.2 Konsolidacija početnih promptova

Ranija skica	Kanonska odluka knjige	Razlog
`secrets.platform.io`	Predloženo `secrets.codepop.tech`	Sopstveni DNS prostor
`injection.enabled + managed`	`injection.mode` + install/grant kontrola	Manje dvosmislenosti
`containerName`	`workloadRefs[].containers`	Eksplicitni potrošači
Globalni `failIfKeyEmpty`	Per-key `nonEmpty`	Nema konfliktnih pravila
`failIfSecretMissing: false`	Nedostajući Secret ne može biti Ready	Jasan ugovor
`maxAge: 30d`	`maxAge: 720h`	Podržana duration semantika
Hash Secret sadržaja	UID/RV metadata token	Bez otiska tajne
Slobodan ESO GVK	Ograničen ExternalSecret adapter	Manji privilegovani opseg

Ovo je konsolidacija **pre implementacije ili prvog objavljivanja**, ne garantovano kompatibilna nadogradnja postojećeg operatora. Ako raniji kod već postoji, prvo uradi inventar, a zatim zasebnu migraciju uz testove i backup specifikacija.

20.3 Predložene razvojne faze

Faza A: jezgro. API, autorizovani read-only scope, validator, status, Secret watch i testovi bez curenja. Rezultat mora raditi bez ESO-a i bez workload write dozvola.

Faza B: upotrebljiv MVP. ESO adapter, workload validacija, rotacioni age signal bez restarta, metrics, Helm, CI i prvi javni release. Obavezni su runbook i jasna podržana verzijska matrica.

Faza C: kontrolisana automatizacija. Zaštićena odobrenja, injection, metadata restart, anti-storm politika i e2e rollout matrica. Ova faza ne sme oslabiti default RBAC profil.

Faza D: širi ekosistem. Više zavisnosti po workload-u, policy report integracije, dodatni workload kontroleri i opciona admission zabrana. Svaka nova integracija mora opravdati povećanu složenost i bezbednosni opseg.

20.4 Doprinosi zajednice

Repozitorijum treba da ima `CONTRIBUTING.md`, `SECURITY.md`, code of conduct, issue template i release proces. Izbor licence se potvrđuje pre javne objave; knjiga ne nameće pravni zaključak o najboljoj licenci.

Dobar bug report sadrži verziju operatora, tip klastera, redigovani ugovor, bezbedan condition izlaz i korake reprodukcije sa sintetičkim Secret-om. Template izričito zabranjuje slanje stvarnih vrednosti i kompletnog kubeconfig-a.

20.5 Pregled PR-a

Za API promenu reviewer proverava default, compatibility i schema. Za validator proverava curenje, determinističnost i budžet. Za mutation proverava autorizaciju, vlasništvo polja, konkurentnost i rollback granicu. Za observability proverava kardinalnost i privatnost metapodataka.

Ne prihvataj feature koji ima samo pozitivan demo. Svaka funkcija mora imati negativan slučaj i dokumentovano ponašanje kada zavisnost nestane. „Radi u mom namespace-u“ nije dovoljan kriterijum za operator zajednice.

20.6 Kada odbiti funkciju

Direktno dobavljanje tajni od provajdera odbija se dok ne postoji snažan razlog za promenu granica proizvoda. Automatsko popravljanje vrednosti odbija se jer operator ne zna nameravani kredencijal. Cross-namespace kopiranje odbija se jer menja model autorstva i izlaganja.

Odbijanje funkcije nije nedostatak ambicije. Mali operator sa jasnim garancijama često je održiviji od platforme koja pokušava da poseduje čitav životni ciklus tajne.

> **Kontrolna tačka.** Svaka roadmap stavka mora navesti koji novi podatak operator čita, koje novo pravo dobija i koji dodatni incident može izazvati.

21. Praktične laboratorije

21.1 Preduslovi i bezbedan opseg

Laboratorija A izvršava se bez klastera. Laboratorije B–E zahtevaju implementiran operator prema specifikaciji knjige, instaliran CRD i namenski test namespace. Ove laboratorije ne tvrde da binarni operator već dolazi u paketu knjige.

Pre svakog cluster testa proveri kontekst. Koristi izdvojen kubeconfig ili kind klaster. Sve vrednosti su test fixture-i. Nijedna komanda ne treba da ispiše environment kontejnera ili sadržaj produkcionog Secret-a.

```
kubectl config current-context
kubectl create namespace sco-lab
```

21.2 Laboratorija A: lokalni validator

U pratećem `examples/contractlab` pokreni testove, race proveru i vet. Dodaj novo pravilo `minLength` za vrednost koja sadrži više-byte Unicode znak. Pokaži testom da je semantika u bajtovima.

Zatim dodaj neispravan URI koji sadrži sentinel i proveri da javni rezultat ne sadrži njegov tekst. Cilj nije samo da validator vrati `false`, već da vrati kontrolisanu šifru i bezbedan rezultat.

Očekivani ishod: testovi prolaze, neispravni slučajevi daju predvidive reason vrednosti i nema Kubernetes zavisnosti.

21.3 Laboratorija B: nedostajući ključ

```
apiVersion: v1
kind: Secret
metadata:
  name: demo-secrets
  namespace: sco-lab
type: Opaque
stringData:
  DB_USERNAME: demo-user
---
apiVersion: secrets.codepop.tech/v1alpha1
kind: SecretContract
metadata:
  name: demo
  namespace: sco-lab
spec:
  secretRef:
```

```
    name: demo-secrets
  requiredKeys:
    - name: DB_USERNAME
    - name: DB_PASSWORD
      minLength: 16
```

Primeni manifest u test klasteru. Očekuj `SecretExists=True`, `KeysValid=False`, `Ready=False` i `DB_PASSWORD` među nedostajućim ključevima. Dodaj sintetički `DB_PASSWORD` kroz fajl namenjen laboratoriji, pa ponovo primeni taj manifest.

Očekivani ishod: Secret watch sam pokreće novi reconcile i ugovor postaje spreman. Nije potreban ručni update ugovora.

21.4 Laboratorija C: ispravna eksplicitna potrošnja

Deployment manifest u pratećim primerima koristi eksplicitni image placeholder za test aplikaciju. Pre izvršavanja zameni ga proverljivim image digest-om sopstvene minimalne aplikacije koja ne ispisuje tajne.

```
spec:
  template:
    spec:
      containers:
        - name: api
          image: YOUR_TEST_IMAGE_BY_DIGEST
          env:
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: demo-secrets
                  key: DB_PASSWORD
```

Dodaj workload referencu sa `containers: [api]`, `consumption: EnvVars` i `requireWorkloadReference: true`. Zatim promeni key referencu na pogrešno ime. Operator mora promeniti `WorkloadsConfigured`, ali ne sme pokušati da automatski ispravi Deployment u validation-only profilu.

Očekivani ishod: nalazi prate stvarnu konfiguraciju potrošača; operator nema workload patch pravo.

21.5 Laboratorija D: zastarela generacija

Sačekaj `Ready=True`, pa promeni ugovor da zahteva novi ključ koji ne postoji. Pokreni gate iz poglavlja 14. Stari Ready condition ne sme zadovoljiti provere za novu generaciju.

Zatim namerno zaustavi operator i ponovi scenario. Gate mora isteći ili prijaviti neuspeh, a ne prihvatiti zastarelo opažanje kao novo. Ponovno pokretanje manager-a treba da nastavi normalno usklađivanje.

Očekivani ishod: generation-aware gate pokazuje vrednost status verzionisanja, ali dokumentacija i dalje priznaje trku između provere i kasnijeg čitanja Secret-a.

21.6 Laboratorija E: rotaciona politika bez restarta

Dodaj pouzdani test timestamp i `maxAge` u ugovor. Koristi injektovani fake clock u testu da prođe vreme bez dugog čekanja. Potvrdi da `RotationPolicySatisfied` prelazi u False čak i kada nema nove Secret promene.

Kada `requireRotationPolicy` nije obavezan, glavni Ready može ostati True; kada postane obavezan, ne sme. Test potvrđuje baš dokumentovanu politiku, ne ono što se korisniku trenutno čini intuitivnim.

21.7 Laboratorija F: opcioni restart

Ova laboratorija pripada tek implementiranoj mutation fazi. Uključi install-level dozvolu, zaštićeno odobrenje i ugovorni restart. Sačuvaj početni PodTemplate token, primeni ispravnu sintetičku Secret promenu i proveri novi token i workload rollout.

Ponovi sa neispravnim sadržajem. Token ne sme da se promeni. Ponovi sa metadata-only promenom; prema politici knjige to može izazvati restart. Na kraju testiraj OnDelete strategiju i očekuj nepodržan režim, ne lažno pozitivan „restart completed“.

21.8 Čišćenje

U test klasteru ukloni laboratorijske ugovore i workload-e, pa namespace. Nemoj uklanjati shared operator CRD iz klastera drugih korisnika. Ako je korišćen namenski kind klaster, njegovo brisanje je jasna granica cleanup-a.

> **Kontrolna tačka.** Laboratorijski izveštaj navodi stvarno izvršene scenarije i rezultate. Fajl primer nije dokaz da je scenario pokrenut.

22. Razvoj uz Codex i kontrola izvršavanja

22.1 Od velikog zahteva do proverljivog zadatka

AI coding agent je najkorisniji kada dobije jasan opseg, prethodne odluke, relevantne fajlove i kriterijume prihvatanja. Zahtev „napravi ceo production operator“ ne definiše šta treba da se proveri niti sprečava da agent preskoči bezbednosne granice.

U dodatku A nalaze se 30 numerisanih promptova. Svaki ima cilj, ograničenja, testove i očekivani izlaz. Promptovi su na engleskom radi rada sa kodom i alatima; objašnjenja knjige ostaju na srpskom. Oni zamenjuju početnu skicu promptova tamo gde je API konsolidovan.

22.2 Zajednički kontekst

Pre svakog zadatka agent čita `AGENTS.md`, kanonsku specifikaciju i bezbednosne odluke. U repozitorijumu se čuva lokalna kopija relevantnih poglavlja, ne oslanja se na to da agent pamti celu prethodnu prepisku.

Kada postoji kod, prvi korak je inventar postojećeg stanja. Agent ne radi novi scaffold preko implementacije, ne menja module path bez odobrene migracije i ne briše unrelated promene. Za svaku razliku između prompta i stvarnog API-ja dokumentuje odluku pre implementacije.

22.3 Nivoi složenosti, ne lažno podešavanje modela

Promptovi imaju preporučeni nivo `Medium` ili `High`. To je oznaka inženjerske složenosti i očekivanog review-a. Nije tvrdnja da Markdown front matter automatski menja stvarni model ili njegov reasoning parametar. Konkretni client i njegove podržane postavke podešavaju se odvojeno.

API, autorizacija, reconcile, mutation i rotacija imaju High oznaku jer zahtevaju razmatranje konkurentnosti i bezbednosti. Dokumentacija i osnovni scaffold mogu imati Medium. Nijedan nivo ne zamenjuje testove i ljudski review.

22.4 Tracking model

Svaki prompt prolazi kroz `pending`, `in_progress`, `blocked`, `implemented`, `verified` i `accepted`. `implemented` znači da kod postoji. `verified` znači da su zahtevane provere stvarno izvršene. `accepted` podrazumeva pregled rezultata i granica.

```yaml
id: "008"
status: pending
depends_on: ["007"]
complexity: High
commit: null
executed_checks: []
not_run: []
reviewer: null
notes: ""
```

Prateći `tracking/progress.csv` i `tracking/README.md` služe kao početni registar. Svi razvojni promptovi su inicijalno `pending`: kreiranje knjige nije izvršavanje tih zadataka. Posebna lokalna Go laboratorija ima zaseban test izveštaj.

22.5 Bezbedno izvršavanje zadataka

Agent podrazumevano radi lokalno. Komande koje menjaju klaster zahtevaju namenski test kontekst, a release publish ostaje eksplicitna aktivnost. Produkcioni kubeconfig i cloud credential-i nisu potrebni za glavninu razvoja.

Nakon svakog zadatka agent izveštava o izmenjenim fajlovima, stvarno pokrenutim komandama, test rezultatima, preostalim rizicima i sledećoj zavisnosti. „Tests should pass“ ne prihvata se kao test rezultat.

22.6 Tačke zaustavljanja

Prva korisna tačka je čisti validator sa status builder-om. Druga je funkcionalni read-only operator koji reaguje na Secret promene. Treća je javni MVP sa dokumentacijom, chart-om, CI-jem i bezbednosnim auditom. Mutation funkcije počinju tek posle prihvatanja tog MVP-a.

Ovaj raspored sprečava da pogodnosti poput injection-a sakriju nerešena pitanja ko uopšte sme da čita tajnu ili menja aplikaciju. Razvojni redosled prati rizik, ne marketinški efekat demonstracije.

> **Kontrolna tačka.** Nijedan prompt se ne označava kao prihvaćen samo zato što je agent završio odgovor. Potrebni su proverljivi commit, izvršeni testovi i review nalaz.

Dodatak A. Trideset detaljnih Codex promptova

Ovaj dodatak je razvojni plan. Promptovi nisu izvršeni stvaranjem knjige. Zajednički kontekst ispod kopira se u `AGENTS.md` ili učitava uz pojedinačni zadatak; svih 30 fajlova u pratećem direktorijumu sadrže sopstveni kontekst.

A.0 Zajednički kontekst

```
You are implementing Secret Contract Operator from the canonical technical book.
Read AGENTS.md, docs/specification.md, docs/security.md and the task dependencies.
Work only within this task. Preserve unrelated changes. Never read production secrets.
Never emit Secret values, fragments, raw parser errors or data hashes into any output.
All references are namespace-local. Mutation is off unless all documented approvals hold.
After work, report changed files, exact executed checks, not-run checks and residual risks.
Update tracking honestly; implementation alone is not verification or acceptance.
```

A.1 Inventar, pravila rada i AGENTS.md

ID 001 · Faza A · Medium · Prethodni zadatak: nema.

```
Inspect the repository before changing files. Preserve an existing module path and any
unrelated work. Create AGENTS.md, docs/specification.md and docs/security.md from the
canonical decisions in this book. Use SecretContract in secrets.codepop.tech/v1alpha1 as the
proposed API, and clearly mark any existing different group as a migration concern.
Define validation-only as both runtime behavior and RBAC. Document the validation-oracle
threat: contract authors must already be authorized to learn properties of the referenced
Secret. Forbid Secret values, substrings, raw parser errors and content hashes in all
outputs. Distinguish application health from contract readiness.
Create tracking records for every numbered task with pending, in_progress, blocked,
implemented, verified and accepted states. Preserve the real current status; do not mark
planned work complete. Record available tools and missing prerequisites without printing
environment variables or kubeconfig contents.

Acceptance: repository inventory and security assumptions exist; no operator implementation
or cluster mutation is performed; tracking starts truthfully.
```

A.2 Kubebuilder scaffold i zaključavanje alata

ID 002 · Faza A · Medium · Prethodni zadatak: 001.

```
Initialize a Go Kubebuilder project only if no scaffold exists. Proposed module is
github.com/CodepopTech/secret-contract-operator, domain codepop.tech, group secrets, version
v1alpha1, kind SecretContract. When a scaffold already exists, inspect and extend it instead
of regenerating over user code.
Select a mutually compatible Kubebuilder, Go, controller-runtime and Kubernetes library set
from the scaffold. Record exact versions in the repository. Do not independently upgrade
```

```
every dependency to latest. Preserve PROJECT, generated Makefile conventions, CRD generation
and the manager entrypoint.
Build the unmodified scaffold, run its relevant tests, make generate and make manifests.
Explain which tests require envtest binaries and whether those were actually available. Add a
reproducible local setup document and ignore local binaries and test credentials.

Acceptance: a clean scaffold builds, generated API/controller files exist, no business logic
is introduced, and actual command results are recorded.
```

A.3 Model pretnji i arhitektonske odluke

ID 003 · Faza A · High · Prethodni zadatak: 002.

```
Write architecture decision records before expanding the controller. Cover namespace-local
references, trusted contract authors, Secret read scope, cache exposure, static safe
diagnostics, no content hashes, no default mutation, no ownership of referenced resources and
no finalizer in the read-only MVP.
Design authorization boundaries explicitly. A user-controlled allow flag or annotation inside
SecretContract is not an authorization mechanism. For the first deployment profile, contract
writers must be trusted administrators with rights to the target Secret. Document a future
protected grant model without claiming that it is already implemented.
Describe a confused-deputy risk for injection into a workload controlled by another actor.
Define the two independent approvals needed before mutation. Document the remaining time-of-
check/time-of-use race and that Ready alone never blocks Kubernetes deployment.

Acceptance: ADRs identify actors, new privileges and failure cases; security claims have
matching planned tests; no unsupported multi-tenant safety promise remains.
```

A.4 Kanonski API tipovi

ID 004 · Faza A · High · Prethodni zadatak: 003.

```
Implement the book canonical API, not the earlier inconsistent sketch. Define local
secretRef, map-like requiredKeys with unique names, optional envName, pointer required/
nonEmpty defaults, byte-based minLength/maxLength, pattern and None/JSON/URI/PEM formats.
Add the bounded ExternalSecret reference, workloadRefs with explicit containers,
includeInitContainers and Any/EnvFrom/EnvVars consumption, injection.mode default Disabled,
and rotation with restartOnChange false, maxAge and rotatedAtAnnotation. Policies are
requireExternalSecretReady, requireWorkloadReference and requireRotationPolicy. Missing
Secret must never satisfy Ready.
Status contains observedGeneration, observedSecret name/UID/resourceVersion, safe violations,
counts, workload results, conditions and lastCheckedTime. Conditions use metav1.Condition and
list-map semantics by type. If optional mutation fields are accepted before implementation,
report FeatureNotEnabled rather than silently pretending to act. Add printer columns backed
by real scalar status fields.

Acceptance: generate DeepCopy and CRD manifests; examples match the types; status subresource
exists; no output type has a Secret value or content fingerprint field.
```

A.5 CRD šema i validacija specifikacije

ID 005 · Faza A · High · Prethodni zadatak: 004.

```
Add structural schema constraints and defaults. Bound requiredKeys to 1..256, pattern size
and workload list sizes. Reject duplicate key names, negative lengths, minLength greater than
maxLength, unsupported kinds and malformed local references. Separate valid Secret key names
from the project conservative environment-name policy.
Require the corresponding reference when an enforcement policy is enabled. Require positive
supported duration syntax such as 720h rather than assuming 30d parses. Reject invalid
annotation keys. Use native schema/CEL where suitable; do not add a mandatory webhook just
for defaults already supported by the schema.
Implement a defensive Go spec validator for cases that cannot safely be enforced in the
chosen schema, including regex compilation. A malformed existing object must become a safe
InvalidSpec result, not panic. Do not copy raw pattern text or parser errors into public
diagnostics.

Acceptance: schema and pure spec tests cover all boundary cases; default false is
distinguishable from omitted default true; generated manifests are clean.
```

A.6 Čisti validator podataka

ID 006 · Faza A · High · Prethodni zadatak: 005.

```
Build internal/contract as a pure Go package with no Kubernetes client, logger, clock or
network access. Adapt the standalone contractlab example rather than importing its
educational module path. The result contains only key identifiers, bounded reason codes and
sorted missing-key lists.
Implement required/optional handling, nonEmpty without whitespace normalization, byte length,
Go regexp semantics, syntactic JSON, absolute URI without network lookup and strict PEM block
validation. Distinguish malformed spec from invalid data. Limit per-value and total input
budgets without truncating values before evaluation.
Do not expose parser errors, values, value lengths, snippets or hashes. Do not claim that
length proves entropy or that parsing proves credential validity. Ensure map iteration cannot
alter public result ordering.

Acceptance: table-driven tests cover positive, negative, optional and budget cases; the
package can be tested independently of a cluster; output serialization is safe.
```

A.7 Fuzz i testovi protiv curenja

ID 007 · Faza A · High · Prethodni zadatak: 006.

```
Expand validator tests using synthetic inputs only. Include invalid UTF-8, Unicode byte
boundaries, malformed URI escapes, PEM bundles, junk before and after PEM blocks, malformed
first PEM blocks followed by valid blocks, regex errors, empty JSON and scalar JSON.
Add deterministic fuzz seeds and a bounded fuzz target checking no panic, deterministic
results and safe result shape. Add sentinel tests for raw, base64, hex and deterministic
digest representations in serialized outputs. Ensure test failures do not dump the full
input.
State explicitly what sentinel testing proves and what it does not prove: it is a regression
test for direct channels, not a proof against a validation oracle. Add an authorized-writer
policy test separately; do not let a logging test substitute for access control.

Acceptance: unit, race and a recorded bounded fuzz run complete where tools support them; any
unrun command is listed as not run.
```

A.8 Deterministički status builder

ID 008 · Faza A · High · Prethodni zadatak: 007.

```
Create a status builder independent of Kubernetes I/O. Encode the exact Ready formula from
the book. Unconfigured optional checks are True/NotConfigured; configured checks can be True,
False or Unknown. Enforced missing configuration is invalid spec.
Set observedGeneration on every condition, preserve meaningful lastTransitionTime and sort
violations/workload results. Clear obsolete findings after recovery. Bound messages and
result lists. Never copy upstream ExternalSecret messages or raw errors.
Do not mutate lastCheckedTime on every no-op reconcile. Define an observation key using
contract generation, authorized dependency metadata and scheduled evaluation triggers. A
status update is necessary only for a relevant new observation or semantic transition. Treat
authorization revocation as a new negative result and remove details no longer authorized for
publication.

Acceptance: repeated identical observations yield an equal status; transitions, stale
generation, error recovery and authorization revocation have direct unit tests.
```

A.9 Osnovni read-only reconciler

ID 009 · Faza A · High · Prethodni zadatak: 008.

```
Implement the first controller using the pure spec/data validators and status builder. Read
the contract, authorize the reference before reading Secret data, fetch the same-namespace
Secret and record UID/resourceVersion for the actual observation.
Handle deleted contracts, missing Secret, Forbidden, API timeout, invalid spec, invalid data
and success as distinct cases. Publish only safe status and bounded events. Use optimistic
status patching and recompute on conflict; do not reuse an old desired status against a new
generation.
Do not mutate Secrets or workloads. Do not add finalizers or ownerReferences to external
objects. Configure event-driven recovery and reasonable transient-error backoff. Ensure an
old positive status is not reported as a newly successful check after a failed read.

Acceptance: envtest covers missing/valid/invalid/deleted Secret and observedGeneration; two
no-op reconciles cause no further status writes.
```

A.10 Indeksi, watch i namespace izolacija

ID 010 · Faza A · High · Prethodni zadatak: 009.

```
Index SecretContract by secretRef.name. Watch non-owned Secret resources and enqueue only
dependent contracts in the same namespace. Add workload/dependency index helpers without
granting ownership of those resources.
Do not apply GenerationChangedPredicate indiscriminately to Secret updates. Filter primary
status-only updates while preserving authorization metadata changes that must trigger
reconciliation. Handle delete and recreate using identity metadata, not name alone.
Write event-driven envtest cases without manually invoking Reconcile to cause the expected
update. Include two namespaces with identically named Secrets, multiple contracts sharing one
Secret, a changed contract reference and index/list failure behavior. Avoid cluster-wide
fallback scans and avoid logging event objects.

Acceptance: create/update/delete events reach only correct dependents; no status feedback
loop or cross-namespace wakeup bug remains.
```

A.11 RBAC profil i cache granice

ID 011 · Faza A · High · Prethodni zadatak: 010.

```
Implement namespace-scoped installation and explicit trusted-writer documentation. Review all
generated RBAC. The default role reads Secret data in the declared scope, updates only
contract status and writes events. It must not write Secrets or workloads.
Document exactly whether the standard cache holds full Secret objects. Do not describe a
label filter as an authorization boundary. If implementing a metadata-watch profile, use
direct authorized reads and test that the cached client cannot silently start a full Secret
informer for that path.
Add negative access tests using the operator ServiceAccount, not the administrator test
client. Separate leader-election Lease permissions in the manager namespace. Clearly reject
or document unsupported arbitrary Secret selection with resourceNames-limited informer lists.

Acceptance: negative tests prove cross-namespace and write restrictions; chart/RBAC output
and runtime watch scope agree; cache exposure is truthful.
```

A.12 Opcioni ESO adapter

ID 012 · Faza B · High · Prethodni zadatak: 011.

```
Add bounded unstructured ExternalSecret integration without importing ESO Go APIs. Limit API
versions to a documented install-level allowlist and keep kind fixed to ExternalSecret. Do
not accept an arbitrary user-controlled GVK.
Support clusters where ESO is absent. Implement discovery plus a documented polling/watch
strategy that still reconciles a referenced ESO object after updates. Check the effective
target Secret name as well as Ready. A healthy ExternalSecret targeting another Secret must
fail with ExternalSecretTargetMismatch.
Sanitize all external conditions to local reason codes. Do not infer credential rotation age
from refreshTime. Test missing CRD, missing object, absent/malformed conditions, false/true
Ready and default/explicit target behavior. Document whether installing ESO after startup
requires rediscovery or restart.

Acceptance: ordinary contracts work without ESO; required ESO failures block Ready; real ESO
compatibility is not claimed until its e2e test is run.
```

A.13 Workload reference validator

ID 013 · Faza B · High · Prethodni zadatak: 012.

```
Implement read-only extraction of PodTemplateSpec from supported apps/v1 workloads. Enforce
explicit named consumers, all-consumer semantics and optional init-container selection.
Missing selected containers fail validation.
Validate EnvFrom, EnvVars and Any according to the canonical API. Respect envName mapping and
required secretKeyRef optional=false behavior. Detect explicit env overrides and ambiguous
later envFrom sources conservatively. Do not read additional unapproved Secret values to
resolve precedence.
Never print conflicting inline env values. Return safe paths and reason codes. Add tests for
sidecars, init containers, mismatched key/name, prefix, optional references, duplicate names
and conflicts. Explain the first-deploy deadlock when requiring an absent workload before
deployment.

Acceptance: all supported consumers are evaluated correctly; no workload mutation occurs;
ambiguous precedence never yields a misleading positive result.
```

A.14 Vremenska rotaciona politika

ID 014 · Faza B · High · Prethodni zadatak: 013.

```
Implement maxAge validation without automatic restarts. Parse the configured annotation as
RFC3339 using an injected clock. Report missing/invalid/future timestamps and expiration with
stable reason codes.
Document that the annotation is a claim by its authorized writer, not proof of credential
rotation. Do not use Secret creationTimestamp, resourceVersion or ESO refreshTime as a
substitute. Use positive supported duration syntax and a documented clock-skew tolerance.
Schedule reconciliation at the next relevant deadline, so expiration is detected without any
object update. Avoid a zero-delay loop after expiry. Apply requireRotationPolicy exactly as
documented; advisory failures may leave Ready true while the dedicated condition is false.

Acceptance: fake-clock tests cover boundary times, skew, invalid dates and restart before
expiry; the next requeue deadline is deterministic and bounded.
```

A.15 Metrike i bezbedni događaji

ID 015 · Faza B · Medium · Prethodni zadatak: 014.

```
Add the documented metric set and safe events. Use bounded result enums, duration units and
explicit Unknown/NotConfigured handling. Per-contract name/namespace labels are an opt-in
cardinality decision; do not add key names, patterns, values, UID or resourceVersion labels.
Emit transition-based events and avoid repeated warnings for identical state. Clear per-
object gauge series when contracts disappear. Ensure multiple manager replicas do not create
misleading dashboards for leader versus standby data.
Secure or scope the metrics endpoint and make ServiceMonitor optional. Add metrics/event
serialization tests using synthetic sentinels. Document health of the manager separately from
contract validity and application availability.

Acceptance: registration is idempotent, deleted-object metrics are removed, event volume is
bounded, and a safe metrics reference document exists.
```

A.16 Envtest integraciona matrica

ID 016 · Faza B · High · Prethodni zadatak: 015.

```
Build a reliable envtest suite around a real manager and watches. Use unique namespaces and
eventual assertions rather than arbitrary sleeps. Install the actual generated CRD schema and
any minimal test-only optional CRDs explicitly.
Cover missing-to-valid-to-invalid Secret transitions, deletion/recreation UID changes,
generation-aware status, authorization revocation, namespace isolation, no-op writes,
concurrency conflicts and timer-driven expiration. Assert status/event/log outputs are free
of sentinels.
Do not wait for Pods to be created by Deployment controllers in envtest because those
controllers are not part of envtest. Reserve actual rollout assertions for kind. Record
downloaded envtest binary versions and make the CI setup reproducible.

Acceptance: tests pass repeatedly from a clean setup without real cloud credentials; flakes,
unrun cases and limitations are documented honestly.
```

A.17 Helm chart za read-only MVP

ID 017 · Faza B · Medium · Prethodni zadatak: 016.

```
Create charts/secret-contract-operator for the implemented MVP. Include image digest/tag
selection, manager deployment, ServiceAccount, namespace-scoped reader roles/bindings, scoped
Lease permissions, resources and hardened security contexts.
Default workload mutation permissions must be absent. Configure metrics and optional
ServiceMonitor without requiring Prometheus Operator. Do not require cert-manager when no
webhook is deployed. Document exact Secret cache and watch scope.
Choose an explicit CRD lifecycle strategy with a separate generated CRD artifact or carefully
documented crds directory usage. Do not claim helm upgrade automatically migrates CRDs. Add
render tests that reject wildcard and forbidden write permissions.

Acceptance: helm lint and template checks pass; CRD and controller installation order is
documented; validation-only RBAC is mechanically verified.
```

A.18 GitHub Actions CI

ID 018 · Faza B · Medium · Prethodni zadatak: 017.

```
Add CI for pull requests and main changes with least privilege. Run format, vet, unit, race,
envtest, generation checks, chart validation and container build. Use exact supported tool
versions and pin third-party Actions to verified immutable commits where appropriate.
Do not give fork pull requests write tokens or cloud credentials. Avoid unsafe
pull_request_target execution of untrusted head code. Treat branch names, titles and other PR
metadata as untrusted input, not shell code.
Fail when generated files differ after make generate/manifests. Keep e2e separated or clearly
configured. Record actual execution and make test output safe for public CI artifacts. Do not
publish an image from ordinary PR checks.

Acceptance: a clean checkout runs the workflow locally where possible; valid YAML and job
permissions are reviewed; no fake Action hashes or secret credentials are committed.
```

A.19 Kind e2e za MVP

ID 019 · Faza B · High · Prethodni zadatak: 018.

```
Create an isolated kind e2e workflow using a pinned node image and local operator image.
Never silently reuse the current user cluster. Require an explicit context and clean up only
resources created by the test.
Install CRDs and the actual chart, wait for manager readiness, create synthetic examples,
verify contract transitions and confirm validation-only RBAC with the actual ServiceAccount.
Restart the manager and verify convergence. Test uninstall without deleting application
Secret/workload resources.
Where ESO integration is advertised, add a scenario with an actual supported ESO release and
a safe fake/test provider. Do not require real AWS/Vault credentials. Capture redacted status
and metadata only; do not archive raw Secret objects.

Acceptance: make e2e has a documented reproducible flow and safe cleanup; results clearly
distinguish executed from planned integration scenarios.
```

A.20 Dokumentacija, primeri i CI gate

ID 020 · Faza B · Medium · Prethodni zadatak: 019.

```
Write README, API reference, security model, troubleshooting, metrics and examples matching
the implemented schema. Include a missing-key example, a valid preflight example, ESO target
validation and workload consumption validation.
Provide a generation-aware gate script that checks contract UID, current generation and
Ready.observedGeneration without reading Secret values. Explain the initial-workload deadlock
and use two deployment phases. Explicitly state the remaining race and that Ready is not
automatic admission enforcement.
Use only synthetic credentials and verified image references or clearly labeled placeholders.
Mark future mutation, webhook and rollout features as future. Ensure every quickstart command
has its prerequisite and expected safe output.

Acceptance: examples validate against the generated CRD in the test environment;
documentation does not advertise unimplemented or untested guarantees.
```

A.21 Bezbednosni audit MVP-a

ID 021 · Faza B · High · Prethodni zadatak: 020.

```
Audit source, tests, chart and docs for leaks and privilege mistakes. Trace every use of
Secret.Data, string conversion, error wrapping, structured logger, event, metric, annotation
and status field. Remove raw object formatting and unsafe upstream messages.
Review the validation-oracle model and ensure contract-writer permissions match declared
trust assumptions. Check authorization before data reads. Verify the default chart cannot
mutate workloads or Secrets. Review pprof/debug exposure, cache contents and public CI
artifacts.
Add regression tests for confirmed issues. Produce SECURITY.md and a concise audit report
with findings, fixes and residual risks. This is an internal engineering audit, not a claim
of independent certification.

Acceptance: no critical known leak or authorization bypass remains; negative tests pass;
residual risks and unperformed external reviews are explicit.
```

A.22 Priprema prvog MVP release-a

ID 022 · Faza B · High · Prethodni zadatak: 021.

```
Prepare, but do not publish without explicit authorization, the first MVP release. Freeze the
v1alpha1 schema, regenerate artifacts, reconcile docs/examples and execute the agreed test
matrix.
Prepare release notes, installation manifests, chart package, checksums, image build
instructions, SBOM/provenance generation and signing verification guidance. Use real
generated digests only; no fabricated release URLs or registry tags.
Document supported versions solely from tested evidence. Separate application version, chart
version and CRD API version. List known limitations including no hard deployment gate,
trusted contract authors and no default mutation. Record release acceptance status in
tracking.

Acceptance: a reviewer can reproduce the local release candidate; unrun checks block
verified/accepted status instead of being silently waived.
```

A.23 Zaštićeno mutation odobrenje

ID 023 · Faza C · High · Prethodni zadatak: 022.

```
Design and implement the authorization boundary for optional mutation after the read-only MVP
is accepted. Require install-level enablement, separate write RBAC, an explicit contract
request and a protected workload/Secret approval tied to contract UID.
Do not treat a freely editable annotation or contract boolean as proof of authorization.
Choose and document an enforceable administrative protection or grant mechanism. If the
necessary protection cannot be provided, leave mutation disabled and mark the task blocked.
Allow at most one mutation-owning contract per workload in the first version. Read-only
contracts may coexist. Recheck approval on every action and immediately stop new actions when
it is revoked. Do not inherit approval on same-name object recreation.

Acceptance: unauthorized, revoked, conflicting and recreated-contract cases cannot mutate the
target; default installation privileges remain unchanged.
```

A.24 Minimalni EnvFrom i EnvVars injection

ID 024 · Faza C · High · Prethodni zadatak: 023.

```
Implement the canonical opt-in injection modes for explicitly selected supported consumers.
Add references only; never materialize Secret values into env.value. Do not mutate init
containers unless the API explicitly supports and tests that path.
Plan changes before writing. EnvFrom adds a non-duplicated reference; EnvVars maps envName or
key name and respects optional rules. Existing conflicting names are reported, not
overwritten. Preserve unrelated env order, image, securityContext and all other fields.
Use concurrency-safe minimal patches and re-read/replan on conflict. Avoid force ownership
takeover. Document partial success across multiple workload objects because Kubernetes does
not provide a multi-object atomic patch. Clarify GitOps field ownership and deletion
behavior.

Acceptance: disabled and unapproved cases make no changes; repeated reconcile is a no-op;
concurrent user updates are preserved; conflict cases have safe status.
```

A.25 Metadata restart token

ID 025 · Faza C · High · Prethodni zadatak: 024.

```
Implement optional restart signaling using only Secret UID/resourceVersion and contract UID.
Use a valid bounded annotation key under the project prefix; never hash Secret content. The
value must distinguish Secret deletion/recreation.
Require all mutation preconditions and valid content before changing PodTemplate. An invalid
new Secret must not trigger a rollout. Document that metadata-only Secret changes may trigger
restart and that first enablement causes an initial template change.
Do not delete Pods directly. Compare the current token before patching. Revalidate the
relevant fresh observation before the action, while documenting that this cannot create an
atomic Secret/workload transaction. Do not claim the annotation proves processes are using
the new value.

Acceptance: tests cover initial enablement, same-version no-op, update, recreation, invalid
data and absence of content fingerprints.
```

A.26 Rollout strategije i sprečavanje oluje

ID 026 · Faza C · High · Prethodni zadatak: 025.

```
Add explicit eligibility checks for supported rollout strategies. Handle paused Deployments,
unsupported Recreate policy if not deliberately allowed, StatefulSet OnDelete/partition cases
and DaemonSet OnDelete. Unsupported cases must not be reported as completed automatic
restart.
Implement a documented debounce and minimum restart interval with persistent/reconstructible
state where needed. Preserve the newest pending observation and revalidate it before
patching. Test manager restart during the waiting window and changes arriving while rollout
is progressing.
Do not claim that a PDB protects every controller-driven rolling update. Do not change the
workload update strategy to make automation convenient. Separate action requested, template
patched and rollout observed states.

Acceptance: fake-clock and concurrency tests prevent repeat storms and lost newest updates;
unsupported strategies are explicit and do not delete Pods.
```

A.27 E2E matrica mutation funkcija

ID 027 · Faza C · High · Prethodni zadatak: 026.

```
Extend kind tests using the actual mutation profile and protected approvals. Verify supported
Deployment rollout, unchanged behavior when disabled, revocation, env conflicts, same-name
Secret recreation and invalid-rotation suppression.
Add supported StatefulSet/DaemonSet cases only when their implementation is complete. Include
OnDelete and partition/paused negative cases. Observe Pod identities and workload rollout
status without exec env or printing credentials.
Test simultaneous GitOps-like changes and the operator patch. Ensure restart storms stop when
mutation is disabled. Confirm uninstall preserves application resources and documents
retained references. Capture only sanitized metadata artifacts.

Acceptance: advertised mutation support is backed by actual e2e evidence; read-only
installation remains least privilege; unexecuted scenarios are not labeled passing.
```

A.28 Opcioni webhook: ADR i uski prototip

ID 028 · Faza D · High · Prethodni zadatak: 027.

```
Evaluate whether any remaining requirement truly needs an admission webhook. Prefer
structural schema/CEL for local spec checks. Keep a hard workload gate as a separate optional
component, not an implied effect of Ready.
Write an ADR covering scope, failure policy, TLS lifecycle, high availability, API call
budget, dry-run, break-glass and dependency cycles. A webhook that only trusts stale Ready
does not provide strong freshness. A live Secret read still does not make later Pod startup
atomic.
Implement only a narrow prototype if these requirements are satisfied and explicitly approved
in project configuration. Otherwise deliver the design and a blocked implementation record.
Default installation must remain usable without cert-manager or admission infrastructure.

Acceptance: no broad cluster-wide blocking behavior appears by default; limitations and
recovery procedure are tested for any implemented prototype.
```

A.29 Kompatibilnost i upgrade testovi

ID 029 · Faza D · High · Prethodni zadatak: 028.

```
Build compatibility tests from actual prior release manifests and documented schema versions.
Do not simulate a nonexistent historical release as evidence. Track served and storage API
versions separately if a new CRD version is introduced.
Test install-old/upgrade-new, defaults, old object readability, safe status transitions and
absence of unintended mutation. Document CRD migration and why Helm rollback alone cannot
guarantee data-schema rollback.
Review all changed condition reasons, defaults, restart-token rules and authorization
semantics as compatibility surfaces. Add migration notes for any earlier sketch field names
only when corresponding real users or code exist.

Acceptance: upgrade evidence identifies exact tested versions; migration risks are explicit;
no destructive CRD deletion is hidden in routine upgrade.
```

A.30 Završni review i release kandidat

ID 030 · Faza D · High · Prethodni zadatak: 029.

```
Perform a repository-wide release review. Recheck API coherence, generation-aware conditions,
namespace authorization, cache exposure, value-free outputs, event cardinality, timer
behavior, idempotent mutation and workload strategy support.
Run the available complete test matrix, generation checks, chart policy checks and container
build. Record exact commands and distinguish failures, blocked prerequisites and not-run
checks. Review every advertised feature against actual code and tests.
Produce a release readiness checklist, residual-risk report, installation/upgrade/uninstall
procedure and prioritized follow-up issues. Prepare signed/reproducible artifacts only from
an accepted commit. Do not publish externally or claim production certification without
explicit user authorization and evidence.

Acceptance: reviewers can trace each public claim to code, tests and documentation; release
acceptance remains blocked for unresolved critical issues.
```

Dodatak B. Sažeta API i reason referenca

B.1 Spec polja

Ovo je normativna projektna referenca knjige. Izvorni Go tipovi i generisana CRD šema treba da budu usklađeni sa njom. Polja planirana za mutation fazu ne znače da je funkcija već dostupna u MVP-u.

Putanja	Tip / default	Pravilo
`secretRef.name`	Obavezni string	Lokalni Secret naziv
`requiredKeys`	Lista 1–256	Jedinstvena po `name`
`requiredKeys[].name`	String	Validan naziv Secret ključa
`requiredKeys[].envName`	Opcion string	Eksplicitno prenosivo env ime
`requiredKeys[].required`	Pointer bool / true	Odsutan obavezni ključ je greška
`requiredKeys[].nonEmpty`	Pointer bool / true	Prazno znači nula bajtova
`requiredKeys[].minLength`	Opcion integer	Minimum dekodiranih bajtova
`requiredKeys[].maxLength`	Opcion integer	Maksimum dekodiranih bajtova
`requiredKeys[].pattern`	Opcion string	Go regexp, ograničena dužina
`requiredKeys[].format`	Enum / None	None, JSON, URI, PEM
`externalSecretRef.name`	Opcion blok, obavezno ime	Fiksni kind ExternalSecret
`externalSecretRef.apiVersion`	Default ESO v1	Iz install-level allowlist-e
`workloadRefs[].apiVersion`	apps/v1	Podržana workload grupa
`workloadRefs[].kind`	Enum	Deployment, StatefulSet, DaemonSet
`workloadRefs[].name`	String	Lokalni workload naziv
`workloadRefs[].containers`	Lista naziva	Svi izabrani moraju proći
`workloadRefs[].includeInitContainers`	false	Eksplicitno uključivanje
`workloadRefs[].consumption`	Any	Any, EnvFrom, EnvVars
`injection.mode`	Disabled	Kasnije EnvFrom ili EnvVars

Putanja	Tip / default	Pravilo
`rotation.restartOnChange`	false	Kasnija odobrena funkcija
`rotation.maxAge`	Opcion duration	Pozitivan, npr. 720h
`rotation.rotatedAtAnnotation`	Projektni ključ	Pouzdani RFC3339 metadata
`policy.requireExternalSecretReady`	false	Obavezan ESO rezultat
`policy.requireWorkloadReference`	false	Obavezna ispravna potrošnja
`policy.requireRotationPolicy`	false	Obavezna starosna politika

B.2 Status polja

`observedGeneration` je generacija proverene specifikacije. `observedSecret` sadrži naziv, UID i resourceVersion stvarno pročitanog Secret-a. `lastCheckedTime` označava relevantnu evaluaciju, ne proizvoljni heartbeat. `missingKeys`, `violations` i njihovi brojači su ograničeni i deterministički.

`workloadStatuses` sadrži identitet i rezultat provere po workload-u i potrošaču. Opcioni action status u kasnijoj verziji odvaja traženje promene od opaženog rollout-a. `conditions` koriste jedinstveni `type`, status True/False/Unknown, observedGeneration, reason, message i lastTransitionTime.

Nikada se ne dodaju polja kao `actualValue`, `sample`, `secretHash`, `valuePreview`, `rawError`, `decodedJSON` ili ceo `Secret` objekat. Metadata identifikatori se takođe ne pretvaraju u neograničene metrics label vrednosti.

B.3 Katalog osnovnih razloga

Reason	Kategorija	Očekivana reakcija
`InvalidSpec`	Konfiguracija	Ispraviti ugovor
`AccessNotApproved`	Autorizacija	Administrativno odobrenje
`SecretNotFound`	Zavisnost	Proveriti naziv i izvor
`SecretAccessDenied`	Infrastruktura	Proveriti RBAC
`MissingRequiredKey`	Podaci	Ispraviti izvor ili naziv
`EmptyValue`	Podaci	Dodati odobrenu vrednost
`TooShort` / `TooLong`	Podaci	Proveriti deklarisanu granicu

Reason	Kategorija	Očekivana reakcija
`InvalidPattern`	Specifikacija	Ispraviti regexp pravilo
`PatternMismatch`	Podaci	Proveriti format izvora
`InvalidJSON`	Podaci	Ispraviti JSON sintaksu
`InvalidURI`	Podaci	Ispraviti apsolutni URI
`InvalidPEM`	Podaci	Ispraviti PEM strukturu
`EvaluationBudgetExceeded`	Budžet	Smanjiti ili opravdati opseg
`ExternalSecretNotReady`	Integracija	Proveriti ESO
`ExternalSecretTargetMismatch`	Integracija	Uskladiti cilj
`ContainerNotFound`	Potrošnja	Ispraviti izbor kontejnera
`EnvConflict`	Potrošnja	Razrešiti vlasništvo env-a
`AmbiguousEnvPrecedence`	Potrošnja	Pojednostaviti izvore
`RotationTimestampMissing`	Metapodaci	Dodati pouzdan signal
`SecretTooOld`	Politika	Pokrenuti odobrenu rotaciju
`FeatureNotEnabled`	Konfiguracija	Ne očekivati neaktivnu funkciju
`UnsupportedRolloutStrategy`	Mutation	Koristiti podržan postupak

Nazivi su projektovani stabilni identifikatori. Implementacija treba da definiše centralne konstante, a ne da iste razloge piše slobodnim string-ovima u više paketa. Dodavanje razloga je API događaj za korisnike alarma i dashboard-a.

B.4 Kontrolna lista release-a

Pre javne objave moraju biti prihvaćeni kanonska šema, bezbednosne pretpostavke, autorizacija, zabrana curenja, status svežina, event-driven oporavak, vremenske provere, RBAC profil i operativni runbook. Potrebni su stvarni unit/envtest/kind rezultati za oglašene funkcije.

Za mutation release dodatno proveri grant opoziv, konflikt vlasništva, očuvanje konkurentnih izmena, invalid-secret suppression, OnDelete/partition slučajeve, anti-storm politiku i uninstall ponašanje. Marketing ne sme podrazumevati automatski restart tamo gde je testiran samo annotation patch.

Dodatak C. Registar arhitektonskih odluka

C.1 ADR-001: lokalne reference

Odluka: Secret, ESO i workload reference ostaju u namespace-u ugovora. **Razlog:** jednostavnija autorizacija i ograničen opseg incidenta. **Posledica:** cross-namespace potrošnja nije podržana; namespace ipak nije dovoljna izolacija za nepouzdane korisnike unutar njega. **Dokaz prihvatanja:** negativni test istog imena u drugom namespace-u.

C.2 ADR-002: pouzdani autori ugovora

Odluka: prvi profil dopušta autorstvo samo subjektima ovlašćenim da saznaju svojstva odgovarajuće tajne. **Razlog:** validacioni rezultat može biti oracle. **Posledica:** MVP nije opšti self-service sistem za nepouzdane tenant-e. **Kasnija mogućnost:** zaštićeni grant model sa ograničenjima pravila i ciljeva.

C.3 ADR-003: read-only nije samo runtime flag

Odluka: default chart nema workload write prava i nikada nema Secret write prava. **Razlog:** programska greška ne treba da iskoristi dozvolu koja default funkciji nije potrebna. **Posledica:** mutation ima zaseban instalacioni profil. **Dokaz:** automatski pregled renderovanog RBAC-a i ServiceAccount negativni testovi.

C.4 ADR-004: rezultat bez sadržaja

Odluka: validator vraća samo kontrolisane šifre, identifikatore ključeva i agregate. **Razlog:** smanjenje direktnih kanala curenja. **Posledica:** parser greške se mapiraju, ne prenose. **Granica:** rezultat i dalje nosi informaciju, pa ne zamenjuje autorizaciju. **Dokaz:** sentinel izlazi i review svih formatiranja.

C.5 ADR-005: metadata token umesto hash-a

Odluka: restart signal koristi Secret UID/RV i UID ugovora. **Razlog:** nema javnog otiska niskoentropijske vrednosti. **Posledica:** metadata-only update može izazvati restart, a inicijalno uključenje menja template. **Granica:** ovo nije dokaz rotacije niti identiteta vrednosti u procesu.

C.6 ADR-006: status bez feedback petlje

Odluka: semantički jednak status se ne upisuje. **Razlog:** manje API opterećenje i stabilnije ponašanje. **Posledica:** vreme provere ima jasno definisan događaj ažuriranja i nije heartbeat svakog reconcile-a. **Dokaz:** test broja status upisa posle stabilizacije.

C.7 ADR-007: Ready nije admission

Odluka: glavni operator održava status, a pipeline ili poseban sloj odlučuje da li deployment sme da nastavi. **Razlog:** jednostavnija dostupnost MVP-a. **Posledica:** eksplicitni generation-aware gate i dokumentovana trka između provere i upotrebe. **Kasnije:** opcioni admission komponentni dizajn sa recovery planom.

C.8 ADR-008: ograničen ESO adapter

Odluka: koristi se fiksni ExternalSecret kind i allowlist verzija, bez direktnog cloud pristupa. **Razlog:** manji privilegovani opseg i jasna odgovornost. **Posledica:** adapter proverava i target naziv; ne obećava podršku za bilo koji future schema oblik. **Dokaz:** odsutan CRD i target mismatch testovi.

C.9 ADR-009: Git-owned env reference

Odluka: osnovni režim očekuje env reference iz Git/Helm konfiguracije. **Razlog:** izbegavanje dva vlasnika istih polja. **Posledica:** managed injection je opcion i zahteva precizno vlasništvo i odobrenje. **Dokaz:** default profil nikada ne patch-uje workload.

C.10 ADR-010: vremenski signal nije dokaz rotacije

Odluka: maxAge računa deklarisanu starost pouzdanog metapodatka, ne dokaz starosti kredencijala. **Razlog:** Kubernetes i ESO metadata ne daju automatski poslovno vreme rotacije. **Posledica:** dokumentuje se ovlašćeni pisac timestamp-a i koristi timer-driven provera.

C.11 ADR-011: bez preuzimanja tuđeg vlasništva

Odluka: nema owner reference preuzimanja ni cleanup finalizer-a za read-only ugovor. **Razlog:** brisanje ugovora ne sme obrisati aplikaciju ili tajnu. **Posledica:** uninstall je jednostavniji, dok kasnije mutation reference mogu ostati prema eksplicitnoj politici. **Dokaz:** delete i uninstall e2e.

C.12 ADR-012: testirani support matrix

Odluka: podržane verzije oglašavaju se samo iz stvarno izvršene matrice. **Razlog:** scaffold kompatibilnost i produkciono ponašanje nisu ista stvar. **Posledica:** release može podržavati manje verzija nego što dokumentacija alata načelno dopušta. **Dokaz:** verzionisan test izveštaj i reproduktivni CI.

Dodatak D. Rečnik i bibliografija

D.1 Rečnik

Admission. Provera ili promena zahteva pre prihvatanja u Kubernetes API.

Condition. Tipizovani signal stanja, sa statusom, razlogom i generacijom.

Contract. Deklaracija očekivanih svojstava tajne i njene potrošnje.

Controller. Proces koji ponavljano usklađuje željeno i opaženo stanje.

CRD. Definicija custom resource tipa u Kubernetes API-ju.

EnvFrom. Grupni izvor environment promenljivih za kontejner.

EnvVars. Projektni naziv režima eksplicitnih secretKeyRef mapiranja.

Generation. Verzija relevantne specifikacije objekta koju kontroler mora opažati.

Grant. Zaštićena odluka o tome ko sme da čita ili menja konkretan cilj.

Idempotentnost. Ponovljena ista operacija nema dodatni neželjeni efekat.

Informer. Mehanizam praćenja resursa i održavanja lokalnog opažanja.

Mutation. Promena resursa, u ovoj knjizi uglavnom PodTemplate reference ili annotation-a.

Observation. Snimak ulaza koje je kontroler stvarno proverio.

Oracle. Izlaz koji omogućava zaključivanje svojstava inače nedostupnog podatka.

Owner reference. Kubernetes veza vlasništva; ne treba je neovlašćeno dodavati tuđem resursu.

PodTemplate. Deklaracija na osnovu koje workload kontroler stvara podove.

Reconcile. Jedno pokušano usklađivanje trenutnog stanja.

ResourceVersion. Identifikator verzije API objekta; ovde se koristi za jednakost.

Rotation. Promena osnovnog kredencijala, ne nužno svaka promena Secret objekta.

Secret. Kubernetes resurs za osetljive konfiguracione podatke.

Status subresource. Odvojeni API put za izveštavanje o stanju custom resursa.

TOCTOU. Razmak između provere stanja i kasnije upotrebe tog stanja.

UID. Identitet jedne životne instance objekta, različit od njenog naziva.

Workload. Resurs koji upravlja izvršavanjem podova, npr. Deployment.

D.2 Primarni izvori

Izvori su provereni 10. oktobra 2026. Dokumentacija sa putanjama `latest` i bez fiksne verzije može se promeniti. Za implementaciju koristi dokumentaciju koja odgovara zaključanim verzijama projekta. Ovi izvori ne predstavljaju audit našeg operatora.

S01 — Kubernetes — Secrets

Nativni Secret objekat i načini upotrebe. Otvoriti zvanični izvor.

S02 — External Secrets Operator — ExternalSecret

Sinhronizacija, target, refreshTime i uslovi. Otvoriti zvanični izvor.

S03 — Kubernetes — Operator pattern

Custom resource i operator obrazac. Otvoriti zvanični izvor.

S04 — Kubebuilder — Introduction

Scaffold i razvoj Kubernetes API-ja. Otvoriti zvanični izvor.

S05 — Kubernetes — API Concepts

Verzionisanje objekata, čitanje i konkurentnost. Otvoriti zvanični izvor.

S06 — controller-runtime — client package

Klijent, reader i optimistički merge patch. Otvoriti zvanični izvor.

S07 — Kubernetes — RBAC Good Practices

Privilegije, Secret čitanje i posredni pristup. Otvoriti zvanični izvor.

S08 — Kubernetes — Good Practices for Secrets

Kontrole pristupa i rukovanje tajnama. Otvoriti zvanični izvor.

S09 — Kubebuilder — Generating CRDs

Markeri, status subresource i generisanje šeme. Otvoriti zvanični izvor.

S10 — Kubebuilder — Configuring envtest

API server i etcd bez punog workload control plane-a. Otvoriti zvanični izvor.

S11 — Go — regexp package

Regexp semantika i linearno izvršavanje. Otvoriti zvanični izvor.

S12 — Go — Fuzzing

Ugrađeno fuzz testiranje. Otvoriti zvanični izvor.

S13 — Kubernetes — Extend API with CRDs

Strukturna šema, default i subresource mogućnosti. Otvoriti zvanični izvor.

S14 — Kubebuilder — Watching Resources

Watch, mapiranje događaja i predicates. Otvoriti zvanični izvor.

S15 — controller-runtime — cache package

Konfiguracija informer cache-a i opsega. Otvoriti zvanični izvor.

S16 — Kubernetes — Define Environment Variables

Kontejnerska env konfiguracija. Otvoriti zvanični izvor.

S17 — Kubernetes — Distribute Credentials Securely

Secret potrošnja i ažuriranje env vrednosti. Otvoriti zvanični izvor.

S18 — Kubernetes — Deployments

PodTemplate promene i rollout strategije. Otvoriti zvanični izvor.

S19 — Kubernetes — StatefulSets

RollingUpdate, OnDelete i partition ponašanje. Otvoriti zvanični izvor.

S20 — Kubernetes — DaemonSet

DaemonSet životni ciklus i ažuriranja. Otvoriti zvanični izvor.

S21 — Kubernetes — Disruptions

PDB i ograničenja pri workload rolling update-u. Otvoriti zvanični izvor.

S22 — Kubernetes — Admission Webhook Good Practices

Dostupnost, opseg i rizici admission kontrole. Otvoriti zvanični izvor.

S23 — Prometheus — Metric and Label Naming

Jedinice, imenovanje i kardinalnost. Otvoriti zvanični izvor.

S24 — Helm — Custom Resource Definitions

CRD instalacija i ograničenja životnog ciklusa. Otvoriti zvanični izvor.

S25 — OpenAI — Custom Instructions with AGENTS.md

Projektne instrukcije za coding agenta. Otvoriti zvanični izvor.

S26 — GitHub — Secure Use of Actions

Dozvole, nepouzdan input i pinovanje Actions. Otvoriti zvanični izvor.

S27 — Kubernetes — Versions in CRDs

Served/storage verzije, konverzija i migracija. Otvoriti zvanični izvor.

D.3 Kako održavati ovo izdanje

Promena implementacije prvo ažurira kanonsku specifikaciju i testove, zatim primere, promptove i dokumentaciju. PDF, EPUB i objedinjeni Markdown moraju se ponovo izgraditi iz istog teksta. Zasebni fajlovi poglavlja služe kao urednički izvor; ručna izmena samo PDF-a nije održiv tok.

Pri sledećem izdanju ponovo proveri datume i ponašanje biblioteka, ali nemoj menjati verzije alata bez testirane matrice. Dodaj changelog knjige za svaku promenu API-ja, bezbednosne garancije ili laboratorijskog koda.